Лабораторная работа №2: Синхронизация потоков в xv6

Оглавление

1. Задача синхронизации 3
   1.1. Гонка и критическая секция 3
   1.2. Требования к решению 4
   1.3. Программные решения: Деккер, Петерсон, Лэмпорт 4
2. Атомарные операции 5
   2.1. Что даёт RISC-V 5
   2.2. Встроенные функции GCC 6
   2.3. Оговорка о порядке памяти 6
3. Блокировки в ядре xv6 7
   3.1. Spinlock 7
   3.2. Sleeplock 8
   3.3. Чего не хватает пользовательскому коду 8
4. Futex 9
   4.1. Переносим sleeplock в пространство пользователя 9
   4.2. Идея futex: проверка значения в ядре 10
   4.3. Синтаксис системного вызова 10
   4.4. История и место futex в Linux 11
   4.5. Как futex устроен в Linux 11
   4.6. Перенос в xv6: проектные решения 12
   4.7. Базовая реализация: глобальная блокировка и sleep/wakeup 13
   4.8. Продвинутая реализация: хеш-таблица очередей 14
   4.9. Бонус: thread_join поверх futex 17
   4.10. За рамками работы 19
5. Примитивы синхронизации поверх futex 19
   5.1. Что мы будем строить 19
   5.2. Как мы будем измерять: futexstats 20
6. Мьютекс 21
   6.1. Определение и свойства 21
   6.2. Мьютекс на двух состояниях 21
   6.3. Что предлагает Дреппер 23
7. Условная переменная 24
   7.1. Определение и синтаксис 24
   7.2. Реализация на счётчике поколений 25
   7.3. Цена broadcast: стадный эффект 27

7.4. Дополнительное задание: FUTEX_REQUEUE 27
8. Семафор 29
8.1. Определение 29
8.2. Паттерны 29
8.3. Интерфейс библиотеки 30
8.4. Базовый уровень: семафор из мьютекса и условной переменной 30
8.5. Продвинутый уровень: семафор прямо на futex 31
8.6. Дополнительное задание: многоразовый барьер 32
9. Блокировка чтения-записи 33
9.1. Задача читателей и писателей 33
9.2. Наивное решение 34
9.3. Голодание писателя 35
9.4. Измерение голодания 35
9.5. Продвинутый уровень: writer-preference 36
9.6. За пределами работы 36
10. Практическая часть 37
10.1. Подготовка 37
10.2. Блок 1. Системный вызов futex 38
10.3. Блок 2. thread_join поверх futex 39
10.4. Блок 3. Мьютекс 40
10.5. Блок 4. Условная переменная 41
10.6. Блок 5. Семафор 42
10.7. Блок 6. Блокировка чтения-записи 42
10.8. Дополнительные задания 43
11. Отчёт 44
12. Контрольные вопросы 44
Литература 45

1. Задача синхронизации

1.1. Гонка и критическая секция

В лабораторной работе 1 мы научили xv6 запускать несколько потоков в одном адресном пространстве, и тест racetest сразу показал обратную сторону этой возможности. Несколько потоков увеличивали общий счётчик, каждый ровно заданное число раз, а итог раз за разом оказывался меньше ожидаемого, причём каждый раз на разную величину. Ничего не «сломалось» — просто инкремент общей переменной не является неделимым действием.

На уровне машинного кода строка counter++ превращается в три инструкции: прочитать значение из памяти в регистр (lw), увеличить регистр (addi), записать регистр обратно (sw). У каждого потока свои регистры, и если два потока выполнят эти тройки вперемешку, один инкремент затрёт другой. На рисунке 1 показан один из таких сценариев: оба потока прочитали 41, оба насчитали 42, оба записали 42 — два инкремента, а счётчик вырос на единицу.

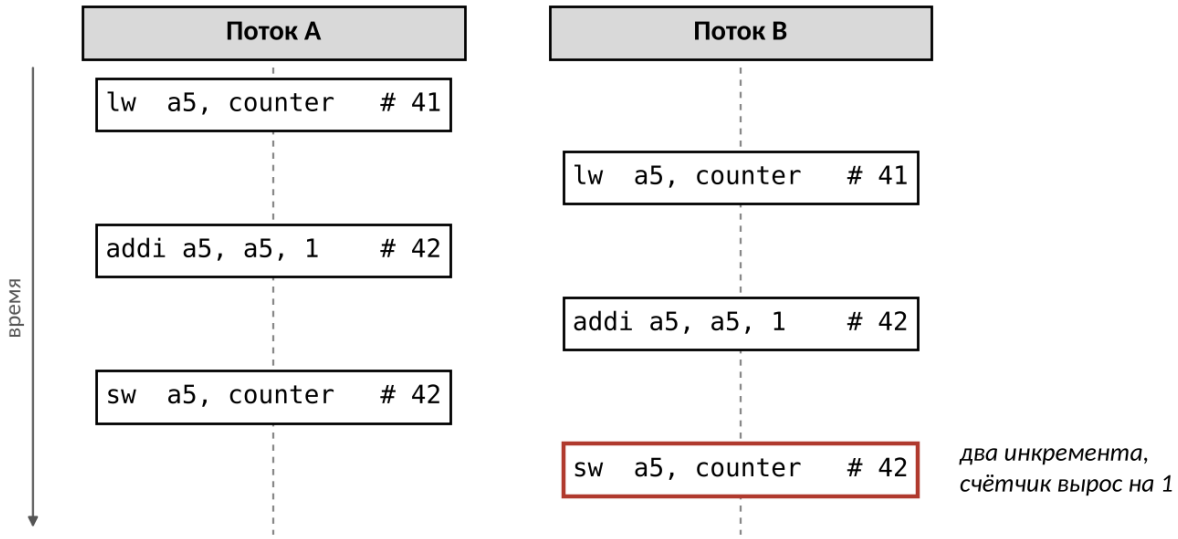


Рисунок 1. Потерянное обновление: чередование инструкций двух инкрементов

Ситуация, в которой результат зависит от того, в каком порядке планировщик и аппаратура перемешали действия потоков, называется **состоянием гонки** (race condition). Участок кода, который работает с разделяемыми данными и потому не должен выполняться несколькими потоками одновременно, называется **критической секцией**.

От состояния гонки стоит отличать более узкое понятие — **гонку данных** (data race). Гонка данных возникает, когда два потока обращаются к одной и той же ячейке памяти, хотя бы одно из обращений является записью, и между этими обращениями нет никакой синхронизации, которая задавала бы их порядок. Пример с counter++ содержит и то и другое: два потока без всякой защиты читают и пишут одну переменную, и из-за этого теряются инкременты. Но понятия не совпадают. Состояние гонки — это ошибка в логике программы: нарушается инвариант, который программист считал верным. Оно возможно и без гонки данных, например, если каждое отдельное обращение к памяти защищено, а последовательность из нескольких таких обращений — нет. Гонка данных, напротив, — это свойство конкретных обращений к памяти, и его можно обнаружить автоматически: именно такие ситуации ищут инструменты вроде ThreadSanitizer. Найти логическую гонку инструмент не может, потому что не знает, какой инвариант программа должна соблюдать.

Задача синхронизации в самой простой постановке — обеспечить, чтобы в критической секции в каждый момент находился не более чем один поток. Всё, что мы будем строить дальше, — мьютексы, условные переменные, семафоры, блокировки чтения-записи — это разные ответы на этот вопрос и на его усложнённые варианты: «не более N потоков», «читатели могут вместе, писатель — только один», «жди, пока не наступит событие».

1.2. Требования к решению

Классически к решению задачи о критической секции предъявляют три требования:

1. **Взаимное исключение.** В критической секции одновременно находится не более одного потока.
2. **Прогресс.** Если секция свободна и в неё хотят войти, решение о том, кто войдёт, принимается за конечное время и только среди желающих; поток, который сейчас не претендует на секцию, не может блокировать остальных.
3. **Ограниченное ожидание.** Поток, заявивший о желании войти, не может пропускать вперёд других бесконечно: число «обгонов» ограничено.

Первое требование — это свойство безопасности (safety): «ничего плохого не происходит». Его нарушение видно в любой момент времени — достаточно застать двух потоков внутри секции. Второе и третье — свойства живости (liveness): «что-то хорошее обязательно произойдёт». Их нарушение нельзя поймать снимком состояния: поток, который ждёт уже минуту, может войти в следующую секунду, а может не войти никогда. Это различие пригодится нам в конце работы: блокировка чтения-записи из раздела 9 будет безупречна с точки зрения безопасности и при этом нарушит свойство живости — писатель будет голодать.

1.3. Программные решения: Деккер, Петерсон, Лэмпорт

Первое корректное решение задачи о критической секции для двух потоков, использующее только обычные чтения и записи памяти, предложил голландский математик Деккер; опубликовал его Дейкстра в 1965 году. Решение получилось громоздким, и в 1981 году Петерсон показал гораздо более компактный вариант. Его стоит увидеть целиком — он помещается в несколько строк:

```
int flag[2];      // flag[i] = 1: поток i хочет войти
int turn;         // чья очередь уступать

void lock(int i) {          // i = 0 или 1
  int j = 1 - i;
  flag[i] = 1;              // я хочу войти
  turn = j;                 // но вежливо уступаю очередь
  while (flag[j] && turn == j)
    ;                       // ждём, пока другой не выйдет или не уступит
}

void unlock(int i) {
  flag[i] = 0;
}
```

Если оба потока одновременно хотят войти, оба поднимают флаги, и каждый записывает в turn номер соседа. Одна из двух записей окажется последней, и она решит спор: войдёт тот, кто уступил первым. Если же хочет войти только один, флаг соседа опущен, и он проходит сразу.

Внимательный читатель вспомнит, что одна строка на Си может превратиться в несколько машинных команд, и спросит, нет ли здесь той же гонки, что с count++. Нет: каждая строка

алгоритма — это либо одно чтение, либо одна запись выровненного слова, а такая операция выполняется целиком, и другой поток увидит либо старое значение, либо новое. Опасной цепочки «прочитать, изменить, записать» здесь нет нигде, и алгоритм специально построен так, чтобы обойтись без неё. Однако у этого кода есть другая опасность, более коварная и связанная не с тем, на какие команды распадаются строки, а с тем, в каком порядке эти команды на самом деле выполняются. К ней мы вернёмся в третьей лабораторной работе.

Для произвольного числа потоков Лэмпорт в 1974 году предложил **алгоритм пекарни** (bakery algorithm): как в булочной, каждый входящий берёт номерок, на единицу больший максимального из уже выданных, и ждёт, пока не обслужат всех с меньшими номерами; совпадения номеров разрешаются по идентификатору потока.

Все эти алгоритмы доказывают важный теоретический факт: взаимное исключение можно построить на одних чтениях и записях. Но на практике ими не пользуются, и причин три:

- Активное ожидание. Ждущий поток крутится в цикле while и сжигает процессорное время. Если владелец секции сам вытеснен планировщиком, ожидающий может прокрутиться весь свой квант впустую.
- Память и время растут с числом потоков. Петерсону нужен флаг на каждого участника, пекарне — номерок на каждого и просмотр всех номерков при каждом входе. Число потоков приходится знать заранее.
- Они не работают на современном железе. Алгоритм Петерсона рассчитан на то, что команды выполняются ровно в том порядке, в каком написаны: сначала поток поднимает свой флаг, и только потом смотрит на флаг соседа. Современные процессоры ради скорости этот порядок не соблюдают. Запись может немного задержаться, а следующее за ней чтение выполнится раньше. Тогда оба потока успевают посмотреть на флаг соседа, пока тот ещё опущен, и входят в секцию одновременно. Так бывает даже на x86. Как с этим бороться, разберём в третьей лабораторной.

Третий пункт — ключевой. Раз без помощи процессора всё равно не обойтись, разумно попросить у него сразу больше: инструкцию, которая читает и изменяет ячейку памяти неделимо. С такими инструкциями взаимное исключение для любого числа потоков строится в две строки.

2. Атомарные операции

2.1. Что даёт RISC-V

Атомарная операция — это действие над ячейкой памяти, которое другие процессоры не могут застать «наполовину выполненным»: они видят ячейку либо до операции, либо после. В архитектуре RISC-V атомарные операции собраны в расширение A, и в нём два семейства инструкций.

AMO (atomic memory operation) выполняют одну операцию «прочитать — изменить — записать» целиком: amoswap.w атомарно записывает в память значение регистра и возвращает старое содержимое ячейки, amoadd.w атомарно прибавляет, есть также amoand, amoor, amomax и другие.

LR/SC (load-reserved / store-conditional) — пара инструкций для операций, которые не укладываются в одну AMO. lr.w читает слово и ставит на его адрес «резервацию»; sc.w записывает новое значение, только если резервация не была нарушена чужой записью, и сообщает в регистре, удалось ли. Если не удалось, последовательность просто повторяют.

Из этой пары собирается, например, сравнение с обменом (compare-and-swap): прочитать, сравнить с ожидаемым, при совпадении попытаться записать.

2.2. Встроенные функции GCC

Писать ассемблерные вставки нам не придётся: GCC предоставляет атомарные операции как встроенные функции семейства __atomic_*. Они доступны и в ядре, и в пользовательских программах xv6 без каких-либо заголовков. Нам понадобятся следующие.

Функция	Что делает	Инструкции RISC-V
__atomic_exchange_n(p, v, __ATOMIC_SEQ_CST)	атомарно записывает v в *p, возвращает старое значение (обмен, xchg)	amoswap.w
__atomic_compare_exchange_n(p, &old, new, 0, __ATOMIC_SEQ_CST, __ATOMIC_SEQ_CST)	если *p == old, записывает new и возвращает 1; иначе записывает текущее значение *p в old и возвращает 0 (CAS)	lr.w / sc.w в цикле
__atomic_fetch_add(p, v, __ATOMIC_SEQ_CST), __atomic_fetch_sub(p, v, __ATOMIC_SEQ_CST)	атомарно прибавляет (вычитает) v, возвращает старое значение	amoadd.w
__atomic_add_fetch(p, v, __ATOMIC_SEQ_CST)	то же, но возвращает новое значение	amoadd.w + сложение
__atomic_store_n(p, v, __ATOMIC_SEQ_CST)	атомарно записывает v в *p	fence + sw

Обратите внимание на интерфейс __atomic_compare_exchange_n. Ожидаемое значение передаётся не по значению, а по адресу, и при неудаче функция перезаписывает его тем, что на самом деле лежало в памяти. Это удобно в циклах повторных попыток: прочитанное значение сразу оказывается под рукой. Но об этом легко забыть и удивиться, что переменная old вдруг изменилась. Четвёртый аргумент, равный нулю, означает «сильный» вариант CAS, который не даёт ложных неудач.

У всех функций семейства последний аргумент (у CAS — два последних) задаёт порядок обращений к памяти. Пока мы везде пишем __ATOMIC_SEQ_CST. Это самый строгий и самый безопасный режим, почему он нужен — в разделе 2.3.

В старых версиях xv6 и во множестве примеров в интернете вы встретите более старое семейство встроенных функций __sync_*: __sync_lock_test_and_set, __sync_val_compare_and_swap, __sync_fetch_and_add, __sync_lock_release, __sync_synchronize. Это предшественники __atomic_*, и соответствие между ними прямое: обмен, сравнение с обменом, атомарное сложение, запись нуля и полный барьер памяти. Аргумента порядка памяти у них нет. Сегодня GCC считает это семейство устаревшим. Читать такой код нужно уметь, но в собственном коде мы будем пользоваться __atomic_*.

Убедиться, во что компилируется каждая функция, легко: после make загляните в листинг user/*.asm или kernel/kernel.asm. Вы увидите у атомарных инструкций суффиксы вроде .aq и .rl. Они относятся к порядку памяти, о котором речь ниже.

2.3. Оговорка о порядке памяти

Здесь необходима оговорка, которую мы пока оставим оговоркой. Атомарность операции и порядок, в котором её результат становится виден другим процессорам относительно соседних обращений к памяти, — это разные вещи. Из раздела 1.3 мы уже знаем, что

процессор может переставлять обычные чтения и записи. RISC-V определяет собственную слабую модель памяти RVWMO, в которой таких перестановок допускается больше, чем на x86.

Аргумент __ATOMIC_SEQ_CST требует самых сильных гарантий. Все такие операции во всех потоках выстраиваются в один общий порядок, с которым согласны все процессоры, а обращения к памяти вокруг них не переставляются через них так, чтобы это нарушило этот порядок. Поэтому код, написанный на таких операциях, корректен без дополнительных размышлений, хотя местами избыточно дорог. В реальных реализациях блокировок обычно указывают более слабые и дешёвые режимы: acquire при захвате, release при освобождении. Именно так устроена спин-блокировка ядра xv6, которую мы разберём в разделе 3.1. Разговор о моделях памяти — отдельная большая тема, к ней мы вернёмся в третьей лабораторной работе. В этой работе достаточно знать, что __ATOMIC_SEQ_CST упорядочивает всё и вопрос закрыт грубой силой.

3. Блокировки в ядре xv6

3.1. Spinlock

Имея атомарный обмен, взаимное исключение получить совсем просто. Заведём слово locked: 0 — свободно, 1 — занято. Захват — это цикл «атомарно запиши 1 и посмотри, что там было»: если было 0, мы первыми подняли флаг и секция наша; если 1 — занято, пробуем снова. Так устроена спин-блокировка ядра xv6 (kernel/spinlock.c), если убрать из неё отладочные проверки:

```
void
acquire(struct spinlock *lk)
{
  push_off();     // запретить прерывания на этом процессоре
  // amoswap.w.aq: атомарный обмен с порядком acquire
  while (__atomic_exchange_n(&lk->locked, 1, __ATOMIC_ACQUIRE) != 0)
    ;             // крутимся, пока не получим 0
  lk->cpu = mycpu();
}

void
release(struct spinlock *lk)
{
  lk->cpu = 0;
  // fence rw,w + sw: все записи секции видны раньше, чем 0 в locked
  __atomic_store_n(&lk->locked, 0, __ATOMIC_RELEASE);
  pop_off();      // вернуть прерывания
}
```

Ядро использует те же функции __atomic_exchange_n и __atomic_store_n, что и мы, но с другим последним аргументом: __ATOMIC_ACQUIRE при захвате и __ATOMIC_RELEASE при освобождении вместо нашего __ATOMIC_SEQ_CST. Это те самые более слабые и дешёвые режимы, о которых шла речь в разделе 2.3; для блокировки их достаточно. Смысл обмена тот же: записать 1 и вернуть старое значение.

Две детали заслуживают внимания. Первая — запрет прерываний на время удержания блокировки. Если бы обработчик прерывания на том же процессоре попытался взять ту же блокировку (а обработчики, например, таймера и консоли это делают), он крутился бы вечно: владелец не может продолжить работу, пока обработчик не закончится. Вторая — порядок памяти acquire и release: он не даёт компилятору и процессору перенести обращения к защищаемым данным за пределы секции.

Спин-блокировка хороша, когда критическая секция коротка — десятки инструкций. Но ждущий поток ничего полезного не делает: он занимает процессор и крутит цикл. Если

секция длинная, например чтение блока с диска, потери огромны, а с запрещёнными прерываниями держать процессор долго и вовсе нельзя.

3.2. Sleeplock

Для длинных секций xv6 использует блокировку со сном (kernel/sleeplock.c). Идея — ждущий поток не крутится, а засыпает, отдавая процессор другим, и его будят, когда блокировка освобождается:

```
void
acquiresleep(struct sleeplock *lk)
{
  acquire(&lk->lk);            // спин-блокировка защищает поле locked
  while (lk->locked) {
    sleep(lk, &lk->lk);        // уснуть на канале lk, отпустив lk->lk
  }
  lk->locked = 1;
  lk->pid = myproc()->pid;
  release(&lk->lk);
}

void
releasesleep(struct sleeplock *lk)
{
  acquire(&lk->lk);
  lk->locked = 0;
  lk->pid = 0;
  wakeup(lk);                  // разбудить всех, кто спит на канале lk
  release(&lk->lk);
}
```

Посмотрите внимательно на второй аргумент sleep. Зачем передавать в функцию засыпания спин-блокировку? Представим, что его нет: поток проверяет locked, видит 1, отпускает lk->lk и только потом засыпает. Между отпусканием блокировки и засыпанием есть окно. Если в это окно другой поток успеет освободить sleeplock и вызвать wakeup, пробуждение уйдёт в пустоту — на канале ещё никто не спит. А наш поток после этого заснёт и не проснётся никогда, хотя блокировка давно свободна. Это классическая проблема **потерянного пробуждения** (lost wakeup), и с ней мы будем встречаться на протяжении всей работы.

xv6 решает её так: sleep(chan, lk) сначала захватывает p->lock — блокировку записи текущего процесса, — и только потом отпускает lk. Функция wakeup, чтобы перевести спящий процесс в состояние RUNNABLE, тоже должна захватить его p->lock. Значит, wakeup не может «проскочить» между проверкой условия и засыпанием: пока ожидающий держит lk, будящий не может изменить условие (он берёт тот же lk в releasesleep), а после того как ожидающий отпустил lk, он уже держит p->lock, и wakeup вынужден ждать, пока тот окончательно не уснёт. Проверка условия и засыпание становятся атомарными относительно пробуждения.

3.3. Чего не хватает пользовательскому коду

Обе блокировки живут в ядре и пользовательским программам недоступны. Спин-блокировку можно было бы повторить и в пользовательском пространстве — атомарный обмен там работает так же, — но с её недостатком: ожидание сжигает процессор, а запретить прерывания и вытеснение пользовательский код не может, так что владелец блокировки может быть вытеснен посреди секции, а все ждущие будут крутиться впустую весь свой квант.

Нам нужен аналог sleeplock для пользовательских потоков, и желательно лучше него. Посмотрите ещё раз на acquiresleep: даже если блокировка свободна и никто не ждёт, мы берём спин-блокировку и вызываем wakeup, проходящий по всей таблице процессов. Для

ядра это терпимо, но для пользовательского кода каждая такая операция означала бы системный вызов. Мьютекс в многопоточной программе берётся миллионы раз в секунду и почти никогда не бывает занят; платить системным вызовом за каждый бесконфликтный захват неприемлемо.

Отсюда требование к механизму, который мы построим в следующем разделе: в отсутствие конкуренции захват и освобождение должны обходиться одной атомарной операцией в памяти пользователя, без входа в ядро; ядро подключается, только когда кому-то действительно нужно уснуть или кого-то действительно нужно разбудить.

4. Futex

4.1. Переносим sleeplock в пространство пользователя

Попробуем сделать для пользовательских потоков то же, что делает sleeplock, но так, чтобы ядро участвовало только при реальной необходимости. Флаг «занято» пусть живёт в обычной памяти процесса — тогда свободную блокировку можно захватить атомарным обменом, вообще не входя в ядро. От ядра нам нужны лишь две услуги, аналоги sleep и wakeup: «усыпи меня на этом адресе» и «разбуди тех, кто спит на этом адресе». Назовём их пока sleep_on(addr) и wake(addr). Ожидающий пишет:

```
while (*addr == BUSY)
  sleep_on(addr);
```

а освобождающий:

```
*addr = FREE;
wake(addr);
```

И сразу попадаем в ловушку из раздела 3.2. Между проверкой *addr и входом в sleep_on есть окно. Если будящий успеет изменить значение и вызвать wake именно в этом окне, пробуждение уйдёт в пустоту — на адресе ещё никто не спит, — а ожидающий после этого заснёт навсегда (рисунок 2).

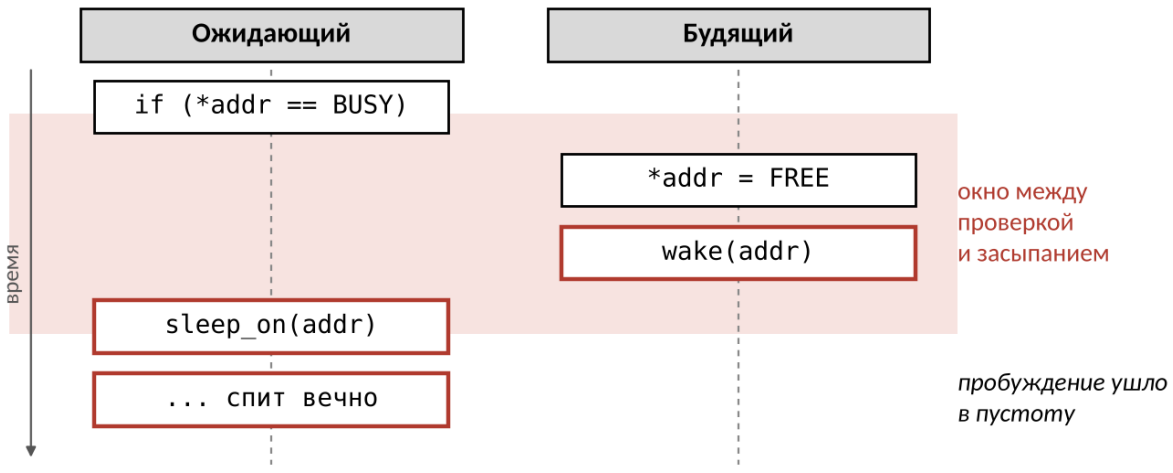


Рисунок 2. Потерянное пробуждение при наивном переносе sleeplock в пространство пользователя

В ядре это окно закрывала спин-блокировка, которую мы передавали вторым аргументом в sleep. Но здесь этот приём не работает: пользовательский код не может передать в ядро спин-блокировку. Захватить спин-блокировку в пользовательском пространстве и отпустить её «изнутри» системного вызова невозможно — ядро про неё ничего не знает, да и держать

спин-блокировку на время системного вызова означало бы заставить всех остальных крутиться в ожидании.

4.2. Идея futex: проверка значения в ядре

Решение, предложенное в 2002 году, изящно: раз в ядро нельзя передать блокировку, передадим ему саму проверку. Вызов «усыпи меня» получает не только адрес, но и значение, которое вызывающий видел по этому адресу, когда решил спать. Ядро под своей внутренней блокировкой — той же, которую берёт пробуждение, — сравнивает текущее содержимое слова с этим значением. Если оно уже изменилось, значит, в окне успело что-то произойти: спать не нужно, ядро немедленно возвращает управление. Если не изменилось — поток ставится в очередь ожидания до того, как внутренняя блокировка будет отпущена, и пробуждение его уже не пропустит (рисунок 3).

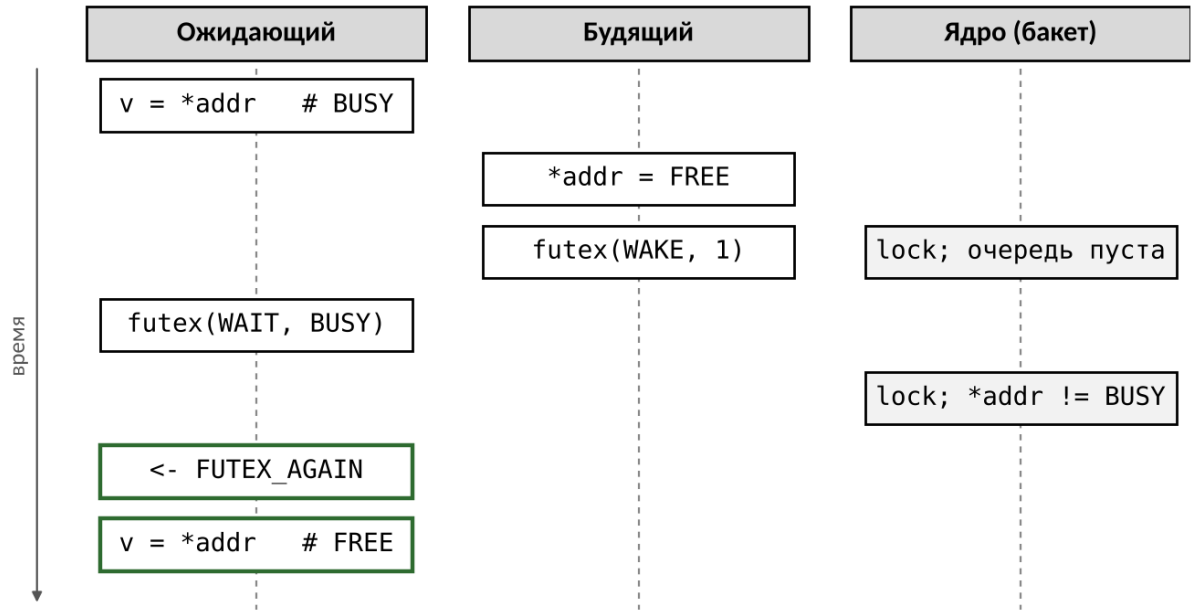


Рисунок 3. Проверка значения в ядре закрывает окно потерянного пробуждения

Получившийся механизм называется **futex** (fast userspace mutex). Он состоит из одного системного вызова с двумя основными операциями:

- **FUTEX_WAIT(addr, val)** — атомарно (относительно FUTEX_WAKE по тому же адресу) сравнить *addr с val; если не равны, немедленно вернуть ошибку, если равны — уснуть до пробуждения;
- **FUTEX_WAKE(addr, n)** — разбудить не более n потоков, спящих на адресе addr, и вернуть, сколько разбудили.

Обратите внимание: значение проверяется дважды. Первый раз — в пользовательском коде, без всяких блокировок: это делает быстрый путь дешёвым, потому что в большинстве случаев ждать не нужно и в ядро идти незачем. Второй раз — в ядре, под блокировкой: это закрывает гонку между решением спать и самим засыпанием.

4.3. Синтаксис системного вызова

В xv6 мы реализуем futex в таком виде:

```
int futex(volatile int *uaddr, int op, int val);
```

Операция	Аргумент val	Возврат
FUTEX_WAIT	значение, которое вызывающий видел в *uaddr перед входом в ядро	0 — спали и были разбужены; FUTEX_AGAIN (−2) — *uaddr != val, сна не было; −1 — плохой адрес
FUTEX_WAKE	сколько потоков разбудить, не больше	число реально разбуженных (может быть 0); −1 — плохой адрес

В Linux вместо FUTEX_AGAIN возвращается ошибка EAGAIN, но в xv6 нет errno, поэтому заводим отдельный код возврата.

Одно свойство синтаксиса нужно запомнить сразу: **возврат 0 из FUTEX_WAIT не гарантирует, что ожидаемое событие произошло**. В Linux ожидание может прерваться сигналом и даже завершиться ложно; в нашей реализации поток, которого убили вызовом kill, тоже вернётся из FUTEX_WAIT с нулём (раздел 4.8). Поэтому FUTEX_WAIT всегда вызывается в цикле с повторной проверкой условия:

```
int v;
while ((v = *addr) == BUSY)        // проверка в пользовательском коде
  futex(addr, FUTEX_WAIT, v);      // повторная проверка - в ядре
```

Цикл обязателен по указанному синтаксису, а не по обстоятельствам конкретной реализации. Удобное следствие: все три исхода FUTEX_WAIT — разбудили, значение изменилось, прервали — обрабатываются одинаково: вернуться к началу цикла и посмотреть на слово ещё раз.

4.4. История и место futex в Linux

До 2002 года основным средством межпроцессной синхронизации в Linux были семафоры System V — тяжёлые объекты ядра, у которых каждая операция, включая захват свободного семафора, была системным вызовом. Хуберт Франке, Расти Расселл и Мэтью Кирквуд в статье «Fuss, Futexes and Furwocks: Fast Userlevel Locking in Linux» (Ottawa Linux Symposium, 2002) предложили перевернуть конструкцию: примитив синхронизации — это обычное слово в памяти пользователя, а ядро подключается только при конфликте. Механизм вошёл в ядро 2.5.7 и стал стандартом с версии 2.6; сегодня на нём построена вся синхронизация библиотеки потоков NPTL (pthread_mutex, pthread_cond, sem_t, pthread_join), а также примитивы многих языковых сред выполнения.

Обратите внимание, futex — не примитив синхронизации, а механизм для их построения. Ядро не знает, что такое мьютекс, условная переменная или семафор. Оно предоставляет ровно одну услугу: очередь ожидания, привязанную к слову памяти, с атомарной проверкой значения при постановке в очередь. Вся семантика — что означает значение слова, когда ждать, кого и сколько будить — определяется библиотекой в пользовательском пространстве. Насколько нетривиальна эта сборка, показывает статья Ульриха Дреппера «Futexes Are Tricky», к которой мы будем обращаться в разделе 6.

4.5. Как futex устроен в Linux

Ожидающих нужно где-то хранить. В Linux это хеш-таблица: массив бакетов futex_hash_bucket, каждый — спин-блокировка плюс список ожидающих. Ключ хеширования вычисляется из адреса, причём двумя способами. Для private-фьютексов (флаг FUTEX_PRIVATE_FLAG, появился в 2.6.22), которыми пользуются потоки одного процесса, ключом служит пара «указатель на адресное пространство + виртуальный адрес»: это дёшево и не требует обхода таблиц страниц. Для shared-фьютексов, видимых из разных процессов через общую память, виртуальные адреса у всех разные, поэтому ключ строится

из физического расположения: страница и смещение в ней. Запомните этот вариант — именно его мы возьмём за образец в xv6.

Элемент очереди, futex_q, содержит ключ, указатель на ожидающую задачу и узел списка. Изящная деталь: futex_q размещается на стеке ядра ожидающего потока. Раз поток заблокирован, его стек ядра гарантированно жив, и динамическое выделение памяти не нужно.

FUTEX_WAIT работает так: вычислить ключ, захватить спин-блокировку бакета, прочитать слово пользователя; при несовпадении — отпустить и вернуть EAGAIN, при совпадении — встать в очередь, отпустить блокировку, отдать процессор. FUTEX_WAKE: вычислить ключ, захватить блокировку бакета, пройти по списку, сравнивая ключи (в один бакет попадают разные фьютексы — коллизии хеша фильтруются точным сравнением), разбудить до n подходящих. Список ожидающих в Linux упорядочен по приоритету, чтобы потоки реального времени просыпались первыми; в xv6 приоритетов нет, и нам хватит очереди FIFO.

4.6. Перенос в xv6: проектные решения

Решение 1: ключ — физический адрес слова

В xv6 в качестве ключа выберем физический адрес слова, получаемый через walkaddr(p->pagetable, uaddr). Для потоков одного процесса он совпадает автоматически: таблица страниц у них общая. Но, в отличие от пары (pagetable, va), он остаётся корректным и для возможных сценариев с разделяемой памятью между процессами. Иными словами, наш futex «от рождения» shared — как ранние фьютексы Linux до появления FUTEX_PRIVATE_FLAG.

Решение 2: слово выровнено на 4 байта

Невыровненный адрес отвергаем с ошибкой. Приятное следствие выравнивания: слово никогда не пересекает границу страницы, так что один результат трансляции адреса описывает его целиком.

Решение 3: ленивая аллокация обрабатывается до блокировок

В нашей ревизии xv6 страницы кучи выделяются лениво, поэтому вполне легальный адрес — например, слово на куче, к которому ещё не прикасались, — может быть не отображён, и walkaddr вернёт 0. Это не ошибка пользователя: страницу нужно догрузить тем же путём, каким это делают copyin и copyout, — вызовом vmfault, — и лишь потом вычислять ключ. Догрузку важно выполнить до захвата каких-либо спин-блокировок: под спин-блокировкой выделять память нельзя.

Решение 4: значение читается прямым разыменованием

Ядро xv6 отображает всю физическую память один в один, поэтому, зная физический адрес, значение слова можно прочитать простым разыменованием. В отличие от copyin, здесь не нужен обход таблиц страниц, и это безопасно делать под спин-блокировкой. Здесь же стоит оценить, от какой сложности нас избавляет простота xv6: страница, однажды отображённая, никуда не денется — свопа и миграции страниц нет. В Linux значительная часть кода futex борется именно с тем, что физическое расположение страницы может измениться, пока на ней кто-то спит.

Вместе эти решения дают две вспомогательные функции, общие для обоих уровней реализации:

```
static uint64
futex_key(uint64 uaddr)
{
  struct proc *p = myproc();
  uint64 pa;

  if(uaddr & 3)                    // выравнивание на 4
    return 0;

  pa = walkaddr(p->pagetable, uaddr);
  if(pa == 0)
    pa = vmfault(p->pagetable, uaddr, 0);   // лениво выделенная страница?
  if(pa == 0)
    return 0;

  // walkaddr/vmfault возвращают базу страницы - добавим смещение
  return pa + (uaddr & (PGSIZE - 1));
}

static inline uint32
futex_load(uint64 pa)
{
  return *(volatile uint32 *)pa;
}
```

4.7. Базовая реализация: глобальная блокировка и sleep/wakeup

Первая реализация собирается из штатных деталей xv6 и занимает пару десятков строк. Один глобальный spinlock futex_lock и уже готовый механизм sleep/wakeup, где каналом сна служит физический адрес слова:

```
int
futex_wait(uint64 uaddr, uint32 val)
{
  uint64 pa = futex_key(uaddr);
  if(pa == 0)
    return -1;

  acquire(&futex_lock);
  if(futex_load(pa) != val){
    release(&futex_lock);
    return FUTEX_AGAIN;
  }
  sleep((void*)pa, &futex_lock);   // канал = физический адрес слова
  release(&futex_lock);
  return 0;
}

int
futex_wake(uint64 uaddr, int n)
{
  uint64 pa = futex_key(uaddr);
  if(pa == 0)
    return -1;

  acquire(&futex_lock);
  wakeup((void*)pa);
  release(&futex_lock);
  return 0;
}
```

Код тривиален, но каждая его строка отвечает на вопрос «почему не иначе». Почему проверка значения и sleep выполняются под тем же локом, который берёт futex_wake? Это и есть решение проблемы потерянного пробуждения из раздела 4.2, спроецированное на xv6. Если wake успел отработать до нашего acquire — мы увидим уже новое значение и вернём FUTEX_AGAIN, спать не будем. Если wake пришёл после — он не сможет пройти дальше своего acquire, пока sleep не уложит нас окончательно: sleep отпускает переданную блокировку только после захвата p->lock, а wakeup для смены состояния нужен тот же p->lock. Цепочка futex_lock → p->lock передаёт атомарность из рук в руки, окна нет. Проведите мысленный эксперимент: уберите acquire из futex_wake (значение-то он не

читает — зачем ему лок?) и найдите чередование, при котором ожидающий засыпает навсегда.

Теперь честно зафиксируем, чего этот уровень не умеет. Во-первых, wakeup(chan) будит всех спящих на канале — аргумент n невыразим, FUTEX_WAKE(uaddr, 1) физически невозможен. Во-вторых, wakeup не сообщает, сколько потоков он разбудил, и возвращать из futex_wake нечего. В-третьих, все фьютексы системы сериализуются одним глобальным локом, а wakeup каждый раз обходит всю таблицу процессов. Для корректности примитивов, которые мы построим дальше, всё это не страшно: благодаря обязательному циклу вокруг FUTEX_WAIT лишние пробуждения безвредны. Но за них придётся платить временем, и в замерах это будет хорошо видно.

4.8. Продвинутая реализация: хеш-таблица очередей

Чтобы будить ровно n потоков, нужно то, чего в базовом уровне нет: явный список ожидающих. Повторяем конструкцию Linux в миниатюре — массив бакетов, в каждом спин-блокировка и FIFO-очередь (рисунок 4).

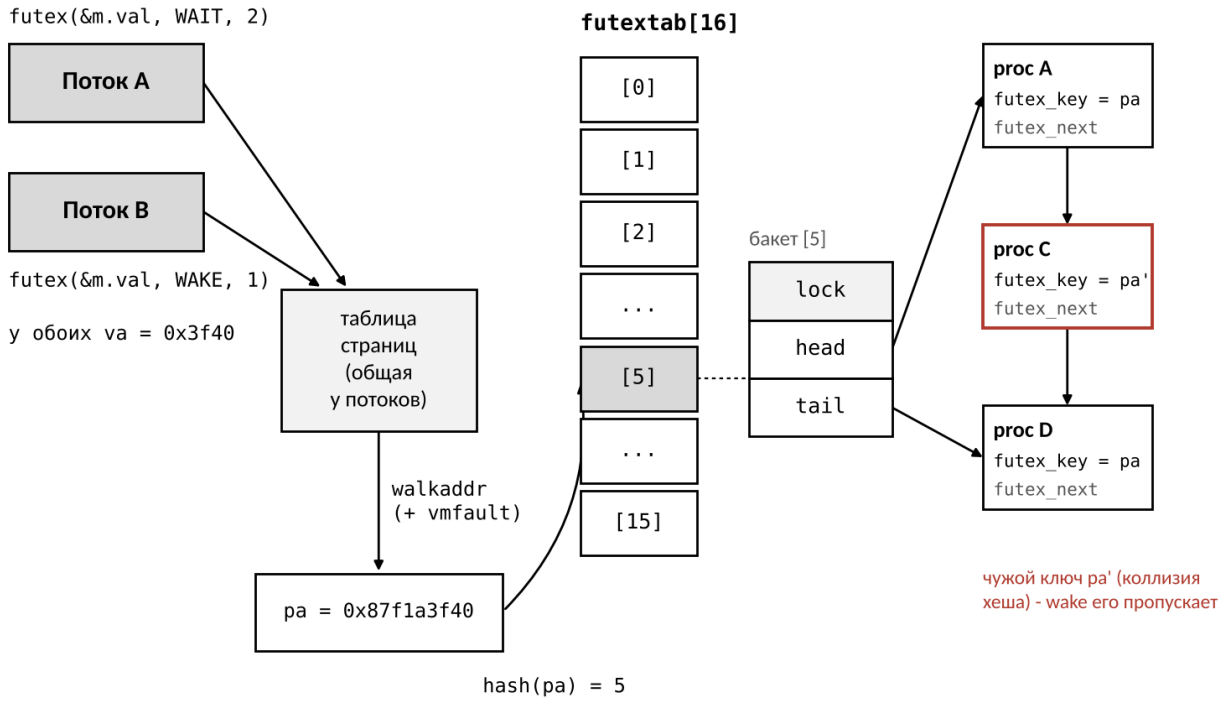


Рисунок 4. Внутреннее устройство futex: от виртуального адреса к очереди ожидающих

```
#define NFUTEXBUCKET 16          // степень двойки

struct futex_bucket {
  struct spinlock lock;
  struct proc *head;             // FIFO: будим в порядке постановки
  struct proc *tail;
};

static struct futex_bucket futextab[NFUTEXBUCKET];

static struct futex_bucket*
futex_bucket(uint64 pa)
{
  return &futextab[((pa >> 2) * 2654435761UL) % NFUTEXBUCKET];
}
```

Бакет выбирается хешем от физического адреса. Младшие два бита у выровненного адреса всегда нулевые — их отбрасываем, иначе три четверти таблицы пустовали бы. Умножение

на нечётную константу (мультипликативный хеш Кнута) перемешивает биты, чтобы соседние слова не толпились в соседних бакетах. В один бакет всё равно попадают разные фьютексы: коллизии неизбежны и лечатся точным сравнением ключа при пробуждении.

Где хранить элемент очереди

Динамическое выделение памяти в ядре — лишняя точка отказа. Вспомним ход Linux: futex_q лежит на стеке ядра спящего потока, потому что время жизни стека заведомо покрывает время ожидания. У нас есть структура с тем же свойством — сама запись proc. Поток может ждать максимум на одном фьютексе, поэтому достаточно двух полей прямо в struct proc:

```
  uint64 futex_key;            // физический адрес слова, на котором спим
  struct proc *futex_next;     // следующий в FIFO-очереди бакета
```

Оба поля защищены блокировкой бакета; не забудьте обнулять их в freeproc.

Ожидание: развёрнутый sleep

Ожидающий не может воспользоваться sleep(chan, lk): просыпаться он будет не через wakeup(chan), а адресно. Придётся развернуть sleep() руками — впервые за курс вы не пользуетесь машинерией sleep/wakeup как чёрным ящиком, а собираете её протокол сами:

```
int
futex_wait(uint64 uaddr, uint32 val)
{
  struct proc *p = myproc();
  struct futex_bucket *b;
  uint64 pa = futex_key(uaddr);
  if(pa == 0)
    return -1;

  b = futex_bucket(pa);
  acquire(&b->lock);
  if(futex_load(pa) != val){       // проверка под блокировкой бакета
    release(&b->lock);
    return FUTEX_AGAIN;
  }
  futex_enqueue(b, p, pa);         // встать в хвост ДО release

  acquire(&p->lock);               // 1) сначала захватить p->lock
  release(&b->lock);               // 2) только потом отпустить бакет
  p->chan = (void*)pa;             // для procdump; матчинг идёт по очереди
  p->state = SLEEPING;
  sched();
  p->chan = 0;
  release(&p->lock);

  acquire(&b->lock);               // нас могли разбудить мимо очереди (kill):
  futex_unqueue(b, p);             // вынем себя сами; после wake это no-op
  release(&b->lock);
  return 0;
}
```

Порядок «захватить p->lock, потом отпустить b->lock» — это дословно приём sleep(): пробуждающему, чтобы вынуть нас из очереди, нужен b->lock, а чтобы сменить наше состояние — p->lock; значит, между постановкой в очередь и уходом в sched() он вклиниться не может. Поменяйте две строки местами — и получите ровно то окно потерянного пробуждения, ради которого всё затевалось. Порядок захвата блокировок b->lock → p->lock одинаков в wait и в wake — нарисуйте себе диаграмму и убедитесь, что взаимная блокировка исключена.

Пробуждение: ровно n потоков

Пробуждающий проходит очередь бакета, фильтруя по ключу, вынимает подходящие элементы и переводит их в RUNNABLE вручную:

```
int
futex_wake(uint64 uaddr, int n)
{
  struct futex_bucket *b;
  struct proc **link, *prev, *pp;
  int nwoken = 0;
  uint64 pa = futex_key(uaddr);
  if(pa == 0)
    return -1;

  b = futex_bucket(pa);
  acquire(&b->lock);
  link = &b->head;
  prev = 0;
  while(*link && nwoken < n){
    pp = *link;
    if(pp->futex_key != pa){     // чужой футекс из того же бакета
      prev = pp;
      link = &pp->futex_next;
      continue;
    }
    futex_unlink(b, link, prev); // вынуть; *link теперь - следующий
    acquire(&pp->lock);
    if(pp->state == SLEEPING){
      pp->state = RUNNABLE;      // адресное пробуждение, минуя wakeup()
      nwoken++;
    }
    release(&pp->lock);
  }
  release(&b->lock);
  return nwoken;
}
```

Перевод в RUNNABLE под pp->lock — это внутренность wakeup(), но без обхода всей таблицы процессов и ровно для n потоков. Обход идёт через указатель на ссылку (link): подходящий элемент вынимается на месте, за один проход по списку. Вспомогательные функции очевидны: futex_enqueue дописывает запись в хвост, futex_unlink вынимает элемент по ссылке на него, поправляя tail, futex_unqueue ищет запись в очереди и вынимает её, если находит.

Незваный будильщик

Осталась самая тонкая часть — поток может проснуться мимо очереди. Функция kkill() переводит процесс из SLEEPING в RUNNABLE, ничего не зная о фьютексах. Отсюда два следствия. Первое: после возврата из sched() поток обязан вынуть себя из очереди сам. Если futex_wake уже сделал это за нас, операция превращается в no-op — поэтому futex_unqueue и написан так, что молча переносит отсутствие элемента. Заодно теперь видно, почему возврат 0 из FUTEX_WAIT не может гарантировать событие: разбуженный kill'ом поток вернётся в пользовательский код, а там цикл перепроверит слово; окончательно судьбу убитого решит usertrap, который проверяет флаг killed после каждого системного вызова. Второе следствие: futex_wake, встретив в очереди поток не в состоянии SLEEPING (уже разбужен kill'ом и ждёт b->lock, чтобы вынуть себя), не должен ни считать его разбуженным, ни пугаться — просто вынимаем и идём дальше.

Почему разыменовывать запись из очереди безопасно

И завершающее обоснование, которое чаще всего пропускают. Между «прочитали указатель из очереди» и «захватили pp->lock» поток теоретически мог бы завершиться, а слот proc — быть переиспользован. Мог бы — если бы не инвариант: последний шаг futex_wait — это unqueue под b->lock. Поэтому, пока мы держим b->lock, ни один поток из очереди не может завершить futex_wait, а значит, не может ни выйти, ни освободить свою запись proc. Членство в очереди плюс удержание блокировки бакета «пришпиливают» запись proc. Сформулируйте этот инвариант в отчёте своими словами — это вопрос защиты.

4.9. Бонус: thread_join поверх futex

Применим futex к задаче, которую мы оставили в лабораторной работе 1 в не самом удачном виде. Там thread_join был системным вызовом, и все ожидающие засыпали на одном общем канале &proc. Функция wakeup(chan) в xv6 проходит по всей таблице процессов и переводит в состояние RUNNABLE каждый поток, спящий на канале chan. Каждый разбуженный дожидается своей очереди у планировщика, получает процессор, проверяет, его ли поток завершился, обнаруживает, что нет, и засыпает снова. При N ожидающих и одном событии мы получаем N−1 бесполезных пробуждений — это **стадный эффект** (thundering herd). А кроме того, даже join уже завершившегося потока стоит системного вызова.

Как это сделано в Linux

Решение NPTL красиво тем, что уведомителем о смерти потока выступает само ядро. При создании потока pthread_create передаёт в clone флаг CLONE_CHILD_CLEARTID и адрес слова ctid в памяти библиотеки. Ядро запоминает этот адрес в дескрипторе задачи. Когда поток завершается — любым способом, включая аварийный, — ядро на пути выхода само записывает 0 по сохранённому адресу и выполняет futex_wake(ctid, 1). Тогда pthread_join сводится к циклу: прочитать слово; если оно уже 0 — поток завершён, выйти без единого системного вызова; иначе futex_wait(&ctid, прочитанное значение) и повторить. Потерянное пробуждение исключено: если поток умер между чтением слова и входом в ядро, слово уже стало нулём, и FUTEX_WAIT вернёт ошибку.

Эта схема закрывает и более тонкую дыру: если бы уведомление посылала библиотечная обёртка в самом завершающемся потоке, поток, убитый до её выполнения, никого бы не разбудил. Перенос записи и пробуждения в ядро гарантирует уведомление при любом сценарии смерти.

Перенос в xv6

Реализуем ту же схему (рисунок 5). В ядре три правки: поле uint64 ctid в struct proc (обнулять в allocproc и freeproc); четвёртый параметр clone — адрес пользовательского слова, сохраняемый в np->ctid; и фрагмент в kexit:

```
if(p->ctid){
  uint32 zero = 0;
  copyout(p->pagetable, p->ctid, (char*)&zero, sizeof(zero));
  futex_wake(p->ctid, 1);
}
```

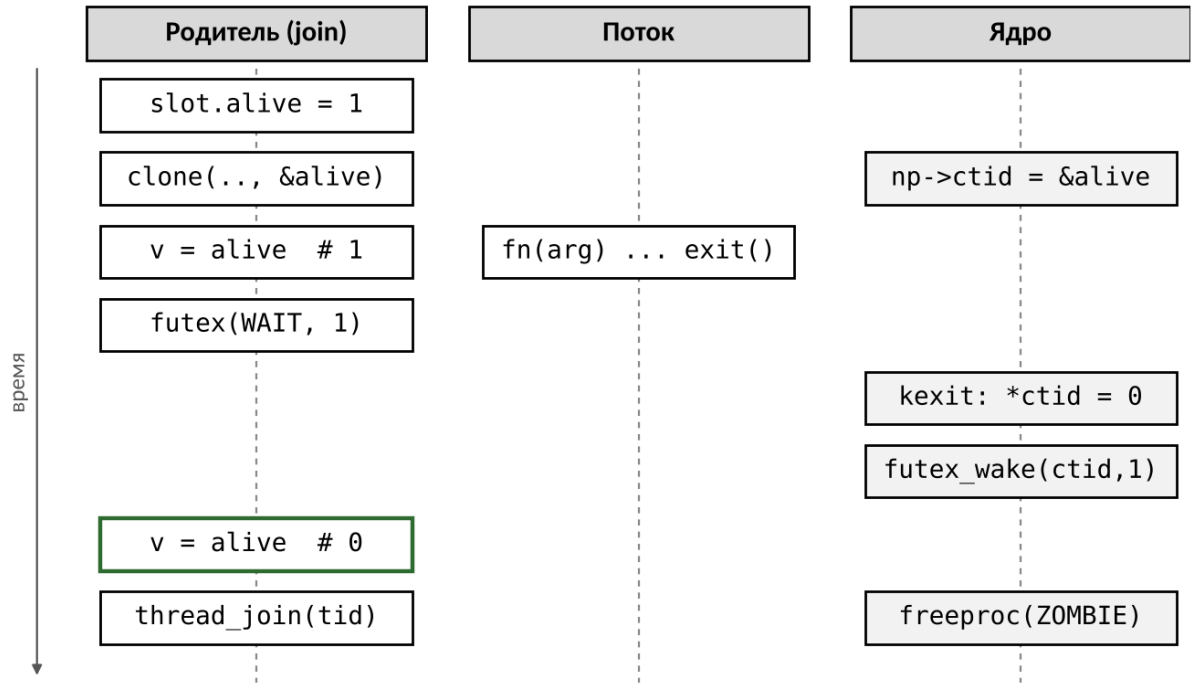


Рисунок 5. Ожидание завершения потока через слово ctid

К этим пяти строкам три вопроса. Почему именно в kexit, а не в библиотечной функции thread_exit? Потому что поток, убитый kill'ом, никакую обёртку не выполнит, а kexit выполняется при любом сценарии смерти. Почему до захвата wait_lock и p->lock? copyout может догружать лениво выделенную страницу, а futex_wake берёт блокировку бакета и p->lock других потоков — делать это, держа свои блокировки, значит выстраивать хрупкую иерархию на ровном месте. Почему таблица страниц здесь ещё жива? Освобождать её (или отвязывать trapframe-слот потока) будет freeproc, который выполнит ожидающий, — заведомо позже.

В пользовательской библиотеке в таблицу потоков добавляется слово volatile int alive — это и есть ctid-слово. thread_create сначала записывает alive = 1 и только потом вызывает clone(fn, arg, stack, &slot->alive). Порядок принципиален: если поставить единицу после clone, поток, успевший завершиться мгновенно, будет затёрт — ядро запишет 0, мы поверх — 1, join повиснет навсегда. (Linux решает ту же гонку зеркально: флагом CLONE_CHILD_SETTID ядро само кладёт tid в слово до старта потока. В glibc в этом слове хранится именно tid, наша единица-«жив» — упрощение.)

thread_join_t сначала ждёт завершения — это чистый pthread_join поверх futex:

```
int v;
while((v = threads[i].alive) != 0)
  futex(&threads[i].alive, FUTEX_WAIT, v);
int r = thread_join(tid);      // убрать останки: запись proc, trapframe
```

Быстрый путь — поток уже мёртв, ноль системных вызовов на ожидание. Медленный — FUTEX_WAIT со значением, которое видели. Затем старый системный вызов thread_join(tid) остаётся в деле, но его роль меняется: он больше не средство ожидания, а сборщик записи ZOMBIE. Почти всегда он отрабатывает мгновенно — к моменту, когда alive стал нулём, поток уже ZOMBIE или станет им через мгновение: есть крохотное окно между записью нуля в ctid и state = ZOMBIE внутри kexit. Если ожидающий проскочит в это окно, thread_join уснёт на общем канале — ради этого редкого случая wakeup(&proc) в

kexit сохраняется. Стадо исчезло не потому, что мы починили канал, а потому, что на нём больше почти никто не спит: ожидание переехало в futex, на индивидуальные слова.

4.10. За рамками работы

У настоящего futex есть ещё несколько операций, которые мы упомянем, но в основной части реализовывать не будем. FUTEX_WAIT с таймаутом — ожидание не дольше заданного срока, основа pthread_mutex_timedlock. FUTEX_REQUEUE и FUTEX_CMP_REQUEUE — разбудить одного, а остальных перевесить в очередь другого фьютекса, не будя; это лекарство от стада в pthread_cond_broadcast, и мы вернёмся к нему в разделе 7.4 как к дополнительному заданию. PI-фьютексы реализуют наследование приоритета для систем реального времени. Robust-фьютексы решают проблему владельца, умершего с захваченным мьютексом. Каждое из этих расширений — отдельная глава инженерной борьбы со следствиями простой идеи «слово памяти плюс очередь в ядре».

5. Примитивы синхронизации поверх futex

5.1. Что мы будем строить

Системный вызов futex — это ещё не примитив синхронизации, а только строительный материал. Сам по себе он не знает, что такое «захватить» или «освободить»: он умеет лишь усыпить поток, если слово в памяти содержит ожидаемое значение, и разбудить заданное число спящих. На основе полученных знаний мы построим четыре классических примитива; каждый следующий опирается на предыдущие (рисунок 6).

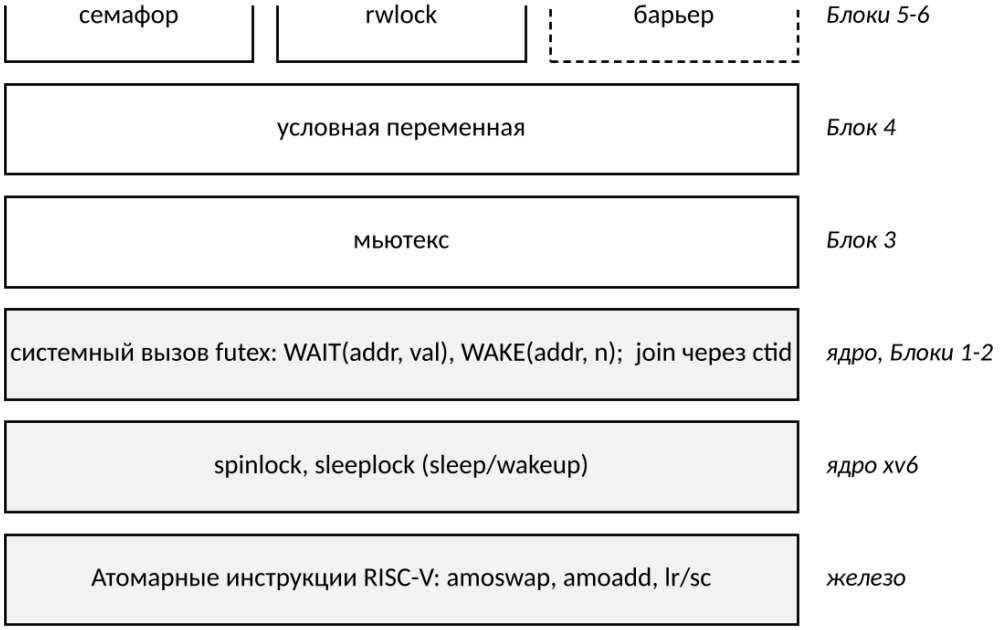


Рисунок 6. Карта работы: от атомарных инструкций к примитивам синхронизации

Примитив	Главный вопрос	Владелец	Хранит состояние	Сколько потоков пропускает
Мьютекс	«кто сейчас работает с данными?»	есть: отпирает тот, кто запер	занят / свободен	одного
Условная переменная	«наступило ли событие?»	нет	нет — событие без ожидающих теряется	столько, сколько разбудили (signal — одного, broadcast — всех)

Примитив	Главный вопрос	Владелец	Хранит состояние	Сколько потоков пропускает
Семафор	«сколько единиц ресурса свободно?»	нет: post может сделать любой	счётчик единиц	столько, сколько единиц в счётчике
Блокировка чтения-записи	«читают или пишут?»	есть	число читателей, флаг писателя	много читателей или одного писателя

Дополнительно, как упражнение, построим барьер — точку, которую ни один из N потоков не проходит, пока до неё не дошли все.

Все конструкции подчиняются одному принципу разделения труда, который уже прозвучал в разделе 3.3. В отсутствие конкуренции операция должна обходиться одной-двумя атомарными инструкциями над словом в памяти процесса, без входа в ядро, — это быстрый путь. Ядро подключается только на медленном пути: когда поток действительно должен уснуть или когда освобождающий знает, что кто-то ждёт и его нужно разбудить. Каждый раз, когда мы будем сравнивать две реализации одного примитива, вопрос будет один и тот же: насколько хорошо она соблюдает этот принцип.

5.2. Как мы будем измерять: futexstats

Утверждения вида «эта версия реже ходит в ядро» нужно подтверждать числами. Для этого ядро ведёт счётчики, а системный вызов futexstats(struct futexstat *s) копирует их снимок в память пользователя.

Счётчик	Где увеличивается	Что показывает
wait_slept	FUTEX_WAIT, поток реально уснул	сколько раз ожидание закончилось сном
wait_again	FUTEX_WAIT вернул FUTEX_AGAIN	сколько раз поток сходил в ядро и не уснул: значение изменилось
woken	FUTEX_WAKE разбудил поток	сколько потоков разбудили (на базовом уровне всегда 0: wakeup не умеет считать)
wake_calls	каждый вызов FUTEX_WAKE	сколько раз сходили в ядро будить, включая вызовы, никого не разбудившие
join_sleeps	засыпание в системном вызове thread_join	сколько раз ожидающие спали на общем канале &proc
requeued	futex_requeue перевесил поток	только для дополнительного задания (раздел 7.4)

Счётчики увеличиваются атомарно (__atomic_fetch_add), без общей блокировки: блокировка на счётчики сериализовала бы все futex-операции системы и обесценила бы бакеты продвинутого уровня. Абсолютные значения смысла не имеют — они копятся с момента загрузки. Каждый замер устроен одинаково: снимок до сценария, снимок после, печать разности. Чтобы не повторять этот код в каждом тесте, в заготовках есть заголовок user/fstat.h с функцией fstat_print:

```
struct futexstat a, b;
futexstats(&a);
/* ... сценарий ... */
futexstats(&b);
fstat_print("mutex solo", &a, &b);
// mutex solo: slept=0 again=0 woken=0 wake_calls=5000
```

Читать вывод удобно через несколько соотношений. wake_calls при отсутствии конкуренции должен быть нулём: если он равен числу операций, реализация ходит в ядро

на каждом освобождении. Большое again относительно slept означает, что потоки массово ходят в ядро и не засыпают — признак слишком часто меняющегося слова. woken, заметно превышающее число реально продвинувшихся потоков, — признак стада.

6. Мьютекс

6.1. Определение и свойства

Мьютекс (mutex, от mutual exclusion — взаимное исключение) — примитив, который защищает критическую секцию. У него две операции: lock (захватить) и unlock (освободить). Пока мьютекс захвачен одним потоком, любой другой поток, вызвавший lock, ждёт. Использование всегда одинаково:

```
mutex_lock(&m);
/* критическая секция: работа с данными, которые защищает m */
mutex_unlock(&m);
```

Несколько свойств мьютекса нужно понимать до того, как писать его реализацию.

У мьютекса есть владелец. Освободить мьютекс должен тот же поток, который его захватил. Наша простая реализация этого не проверяет (как и большинство быстрых реализаций), но нарушение этого правила — ошибка в программе: другой поток, отперев чужой мьютекс, впустит в секцию второго и сломает взаимное исключение.

Мьютекс не рекурсивен. Если поток, уже владеющий мьютексом, вызовет lock повторно, он будет ждать самого себя — вечно. В POSIX есть рекурсивные мьютексы, которые считают глубину захвата, но это отдельный тип с отдельной ценой.

Мьютекс не обещает справедливости. Когда владелец освобождает мьютекс, а несколько потоков ждут, никто не гарантирует, что войдёт тот, кто ждал дольше всех. Более того, поток, только что подошедший к мьютексу, может захватить его быстрым путём раньше, чем проснётся разбуженный ожидающий. Это называется перехватом (barging). Он снижает справедливость, но повышает пропускную способность: мьютекс не простаивает, пока разбуженный поток добирается до процессора.

Ожидать события под мьютексом нельзя. Мьютекс отвечает на вопрос «кто сейчас работает с данными», но не на вопрос «изменились ли данные так, как мне нужно». Если поток, держа мьютекс, будет ждать, пока в очереди появится элемент, производитель не сможет войти в секцию и элемент не появится никогда. Для ожидания событий служит условная переменная (раздел 7).

6.2. Мьютекс на двух состояниях

Вернёмся к сюжету раздела 4.1. Мы хотели перенести sleeplock в пространство пользователя и упёрлись в потерянное пробуждение. Теперь у нас есть futex, и перенос получается дословным: флаг «занят» — слово val, сон — FUTEX_WAIT, пробуждение — FUTEX_WAKE.

```
struct mutex {
  int val;            // 0 - свободен, 1 - занят
};

void
mutex_xchg_lock(struct mutex *m)
{
  while (__atomic_exchange_n(&m->val, 1, __ATOMIC_SEQ_CST) != 0)
    futex(&m->val, FUTEX_WAIT, 1);
}

void
mutex_xchg_unlock(struct mutex *m)
{
  __atomic_store_n(&m->val, 0, __ATOMIC_SEQ_CST);
  futex(&m->val, FUTEX_WAKE, 1);
}
```

Захват — атомарный обмен: записать 1 и посмотреть, что там было. Если 0 — мьютекс был свободен, и мы его захватили одной инструкцией, не входя в ядро. Если 1 — он занят; засыпаем на слове с ожидаемым значением 1. Освобождение записывает 0 и будит одного ожидающего.

Почему нет потерянного пробуждения

Пусть поток A держит мьютекс, а поток B выполнил обмен, получил 1 и собирается вызвать FUTEX_WAIT(1). Если A успеет освободить мьютекс раньше, чем B войдёт в ядро, слово станет 0, и проверка в ядре вернёт B FUTEX_AGAIN. B вернётся в цикл, снова выполнит обмен — и на этот раз получит 0, то есть захватит мьютекс. Если же A освободит мьютекс после того, как B встал в очередь, FUTEX_WAKE его разбудит. Третьего не дано: проверка значения и постановка в очередь атомарны относительно пробуждения.

Заметьте ещё одно свойство: пробуждение не передаёт владение. Разбуженный поток не считает себя владельцем, а снова выполняет обмен. Владение всегда устанавливается атомарной операцией, а пробуждение — лишь сигнал «попробуй ещё раз». Поэтому все исходы FUTEX_WAIT сходятся в одну точку цикла, и перехват (раздел 6.1) здесь естественен: пока разбуженный B добирается до процессора, третий поток C может захватить мьютекс обменом, и B просто уснёт снова.

Почему ожидающие не мешают друг другу

Все ожидающие пишут в слово одно и то же значение 1. Пока мьютекс занят, слово стабильно равно 1, и FUTEX_WAIT(1) у каждого ожидающего засыпает с первой попытки. Это важное свойство, и в разделе 6.3 мы увидим вариант, где оно нарушено.

Дефект

У этого мьютекса один дефект, и он хорошо измеряется: mutex_xchg_unlock не знает, ждёт ли кто-нибудь, и поэтому на каждом освобождении делает системный вызов. Поток, который тысячу раз захватил и отпустил никем не оспариваемый мьютекс, тысячу раз сходил в ядро, взял блокировку бакета, убедился, что очередь пуста, и вернулся ни с чем. Весь смысл futex — держать быстрый путь в пространстве пользователя — потерян ровно наполовину: захват дёшев, освобождение — нет. В замере это фиксирует счётчик wake_calls: на прогоне из 5000 захватов без конкуренции он равен 5000.

Чтобы unlock мог не ходить в ядро, он должен знать, есть ли ожидающие. А для этого ожидающие должны сообщать о себе — через то же слово.

6.3. Что предлагает Дреппер

Ульрих Дреппер, автор NPTL, в статье «Futexes Are Tricky» последовательно разбирает три реализации мьютекса на futex — Take 1, Take 2 и Take 3 (разделы 4–6 статьи). Это обязательное чтение для продвинутого уровня: вам предстоит разобрать варианты по статье, реализовать их и сравнить замерами с нашим мьютексом на двух состояниях. Здесь мы только обозначим идеи.

Take 1: инкремент

Первая версия Дреппера кодирует в слове не флаг, а счётчик: захват — атомарный инкремент, и если старое значение было не нулём, поток засыпает с ожидаемым значением «старое + 1». Освобождение пишет 0 и безусловно будит одного. Версия корректна, но у неё два дефекта. Первый — тот же, что у нашего мьютекса: unlock всегда ходит в ядро. Второй — тоньше и связан как раз со свойством «ожидающие не мешают друг другу», которое у нас выполнено, а у Take 1 нет. Найдите этот дефект сами; подсказка: что происходит со словом, когда два потока почти одновременно обнаруживают мьютекс занятым? Дреппер описывает и третью проблему этой версии — переполнение счётчика.

Take 2 и Take 3: три состояния

Главная идея следующих версий — различать не два, а три состояния слова:

Значение	Смысл	Что делает unlock
0	свободен	—
1	занят, ожидающих нет	переводит в 0, системного вызова нет
2	занят, есть (или могут быть) ожидающие	переводит в 0 и будит одного: FUTEX_WAKE(1)

Захват свободного мьютекса — по-прежнему одна атомарная операция (CAS 0→1). Поток, обнаруживший мьютекс занятым, обязан перед сном перевести слово в состояние 2, чтобы будущий unlock знал, что нужно идти в ядро. Освобождение — атомарный декремент: если старое значение было 1, мьютекс уже свободен и ожидающих не было — вот исчезнувший системный вызов; если 2 — кто-то ждёт, нужно дописать 0 и разбудить. Переходы между состояниями показаны на рисунке 7.

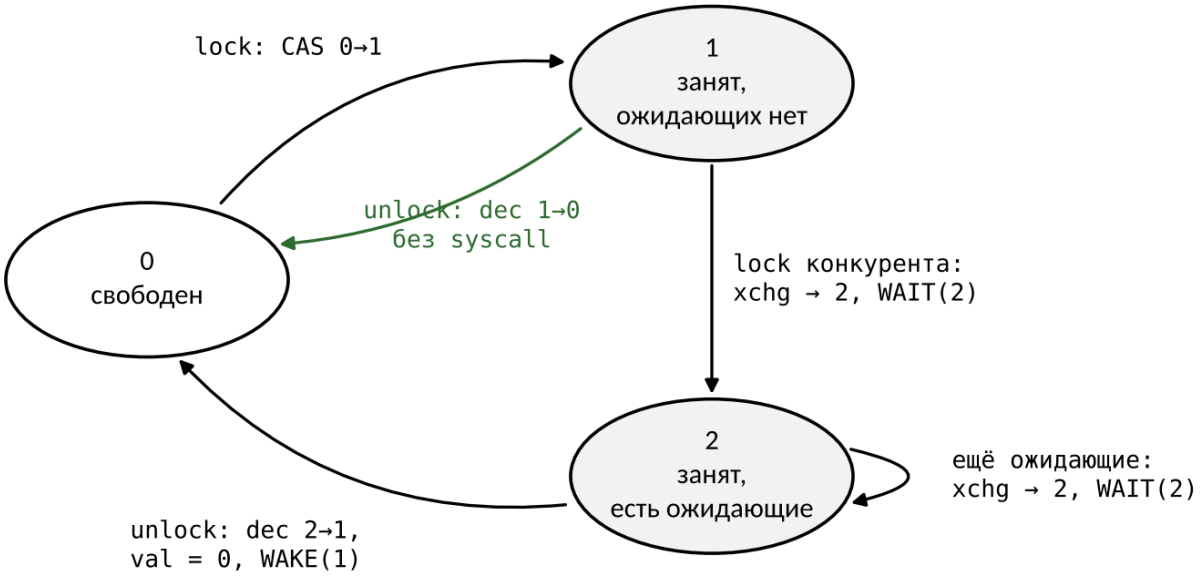


Рисунок 7. Автомат состояний мьютекса на трёх состояниях

Take 2 и Take 3 отличаются тем, как именно ожидающий переводит слово в 2: в Take 2 это делается сравнением с обменом, в Take 3 — атомарным обменом xchg. Разберитесь по статье, почему оба варианта корректны, чем Take 3 проще и что означает ситуация, когда обмен на 2 вернул 0. Ответ на последний вопрос — ключ к пониманию всей конструкции, его нужно будет изложить в отчёте.

Спин перед сном

Наконец, есть соображение, не меняющее протокол: если критические секции коротки, мьютекс, занятый прямо сейчас, скорее всего освободится через десятки-сотни тактов — быстрее, чем поток успеет уснуть и проснуться через ядро. Поэтому перед тем, как переходить к тяжёлой артиллерии, стоит немного покрутиться в пространстве пользователя, повторяя попытку быстрого пути. Важная деталь: крутящийся поток не должен переводить слово в 2 — иначе владелец при освобождении увидит 2 и пойдёт в ядро будить того, кто и так не спит. Это та же идея, что у адаптивных мьютексов glibc: оптимизм на короткой дистанции, сон на длинной. Число попыток подбирается экспериментально, и оба режима — короткую и длинную критическую секцию — стоит показать замером.

7. Условная переменная

7.1. Определение и синтаксис

Мьютекс не умеет выражать ожидание события: «в буфере появился элемент», «очередь опустела», «все рабочие закончили». Ждать события, удерживая мьютекс, нельзя (раздел 6.1). Ждать, отпустив мьютекс и проверяя условие в цикле с повторным захватом, можно, но это опрос, который сжигает процессор. **Условная переменная** (condition variable, condvar) — примитив, который позволяет атомарно «отпустить мьютекс и уснуть до сигнала», а проснувшись — снова захватить мьютекс.

У условной переменной три операции:

- **wait(cv, m)** — вызывается под мьютексом m: атомарно отпускает m и засыпает; проснувшись, снова захватывает m и только потом возвращается;

- **signal(cv)** — разбудить одного из ожидающих (если они есть);
- **broadcast(cv)** — разбудить всех ожидающих.

Условная переменная **не хранит состояние**. Сигнал, поданный, когда никто не ждёт, просто теряется — в отличие от семафора, который его запомнит (раздел 8). Само «условие» живёт не в переменной, а в данных программы, защищённых мьютексом, — это предикат вроде count > 0. Отсюда жёсткое правило использования, обязательный для любой реализации:

```
mutex_lock(&m);
while (!predicate)            // while, а не if!
  cond_wait(&cv, &m);
/* предикат истинен, мьютекс наш */
mutex_unlock(&m);
```

а на стороне того, кто меняет данные:

```
mutex_lock(&m);
/* изменить данные так, что предикат может стать истинным */
cond_signal(&cv);             // или cond_broadcast
mutex_unlock(&m);
```

Почему while, а не if? Потому что пробуждение не означает истинности предиката. Поток могли разбудить сигналом, адресованным логически другому событию, охраняемому той же переменной. Broadcast будит всех, а данных может хватить одному: первый проснувшийся заберёт элемент, остальные обнаружат пустую очередь. Между сигналом и моментом, когда разбуженный поток снова захватит мьютекс, в секцию может успеть войти третий поток и изменить данные. Наконец, POSIX явно разрешает ложные пробуждения (spurious wakeups) — возврат из wait вообще без сигнала. Цикл while делает все эти случаи безвредными: раз пробуждение — лишь совет «перепроверь», вся логика корректности сосредоточена в предикате, который проверяется под мьютексом.

Выбор между signal и broadcast — вопрос не стиля, а того, скольких ожидающих способно пропустить событие. Положили в буфер один элемент — достаточно разбудить одного потребителя. Объявили, что работа закончена и все должны выйти, — нужен broadcast.

7.2. Реализация на счётчике поколений

Наша условная переменная — одно слово, беззнаковый счётчик поколений seq. Каждый сигнал — это инкремент счётчика; спящие спят на его текущем значении, и любое изменение для них означает «что-то произошло, проверь предикат». Реализация в этой работе дана целиком — ваша задача понять, почему она правильна.

```
struct cond {
  unsigned seq;
};

void
cond_wait(struct cond *cv, struct mutex *m)
{
  unsigned old = cv->seq;          // читаем поколение, ещё держа мьютекс
  mutex_unlock(m);
  futex((int *)&cv->seq, FUTEX_WAIT, (int)old);
  mutex_lock(m);
}

void
cond_signal(struct cond *cv)
{
  __atomic_fetch_add(&cv->seq, 1, __ATOMIC_SEQ_CST);
  futex((int *)&cv->seq, FUTEX_WAKE, 1);
}

void
cond_broadcast(struct cond *cv)
{
  __atomic_fetch_add(&cv->seq, 1, __ATOMIC_SEQ_CST);
  futex((int *)&cv->seq, FUTEX_WAKE, 0x7fffffff);
}
```

Порядок действий в cond_wait не случаен, и в нём сосредоточена вся корректность. Значение поколения читается **до** освобождения мьютекса, то есть в мире, согласованном с предикатом: пока мы держим мьютекс, никто не может изменить защищаемые данные и подать сигнал. Классическая гонка потерянного пробуждения выглядела бы так: потребитель проверил предикат (буфер пуст), отпустил мьютекс, и в этот миг производитель положил элемент и подал сигнал, а потребитель после этого заснул навсегда, потому что сигнал пришёлся в никуда. С futex эта гонка закрыта конструктивно (рисунок 8): сигнал увеличил seq, и когда опоздавший потребитель доберётся до FUTEX_WAIT с устаревшим old, ядро под блокировкой бакета увидит несовпадение и вернёт FUTEX_AGAIN, не усыпляя. Окно между «решил спать» и «уснул» существует всегда, но futex делает его безвредным.

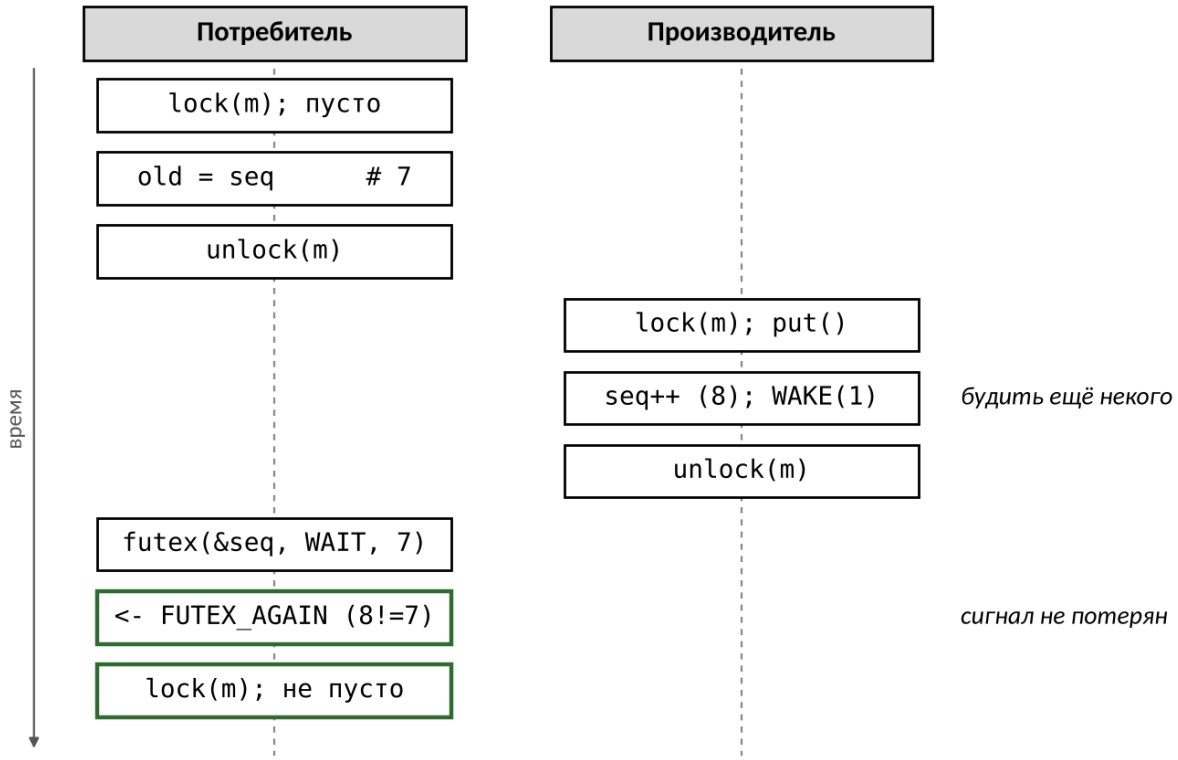


Рисунок 8. Сигнал, поданный между unlock и FUTEX_WAIT, не теряется

Счётчик поколений беззнаковый и рано или поздно переполнится. Здесь это безобидно: нам не важно значение счётчика, важен лишь факт его изменения между чтением old и проверкой в ядре, а арифметика по модулю 2^{32} этот факт сохраняет. Теоретическая гонка ровно в 2^{32} сигналов за время между двумя инструкциями потока в учебной системе может считаться несуществующей.

7.3. Цена broadcast: стадный эффект

У cond_broadcast в этой реализации есть системный порок. Все разбуженные потоки, выйдя из FUTEX_WAIT, немедленно выстраиваются в очередь за одним и тем же мьютексом: первый его захватывает, остальные проваливаются в FUTEX_WAIT уже на слове мьютекса. Мы разбудили N потоков, чтобы N−1 из них тут же уснули снова, — это тот же стадный эффект, что был у thread_join на общем канале, только теперь он возникает на уровень выше, из семантики самого broadcast. Тест herdbcast показывает его в чистом виде: восемь потоков ждут на условной переменной, главный поток под мьютексом делает один broadcast и ещё немного держит мьютекс. Результат — slept=8: каждый разбуженный уснул повторно.

Подчеркнём, что это вопрос производительности, а не корректности: все восемь в итоге по очереди пройдут, и предикат каждый проверит под мьютексом.

7.4. Дополнительное задание: FUTEX_REQUEUE

Взрослое решение проблемы — операция FUTEX_REQUEUE (и её безопасная наследница FUTEX_CMP_REQUEUE): разбудить одного, а остальных, не пробуждая, **перевесить** из очереди ожидания на слове условной переменной в очередь на слове мьютекса. Тогда спящие проснутся не все разом, а по одному, по мере того как мьютекс будет переходить из рук в руки, — стадо превращается в организованную очередь (рисунок 9). Именно так glibc реализовывала pthread_cond_broadcast много лет.

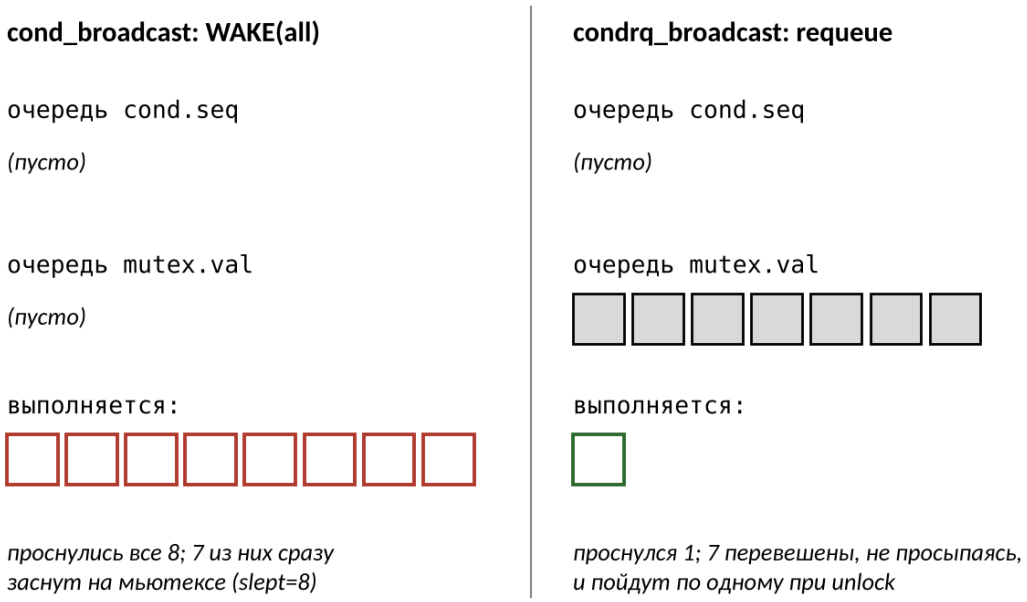


Рисунок 9. broadcast без requeue и с requeue

Задание состоит из двух частей. В ядре — новый системный вызов:

```
int futex_requeue(volatile int *uaddr, int nwake,
                  volatile int *uaddr2, int nmove, int cmpval);
```

Если *uaddr == cmpval, разбудить до nwake потоков, ждущих на uaddr, и ещё до nmove перевесить в очередь uaddr2; вернуть сумму разбуженных и перевешенных. Если значение не совпало — вернуть FUTEX_AGAIN, ничего не трогая (это и есть «CMP»: если между инкрементом seq и вызовом пришёл ещё один сигнал, вызывающий должен перечитать состояние). Бакетная архитектура позволяет сделать это небольшим усилием, но с тремя тонкостями, которые и составляют суть задания:

1. Блокировок теперь две — бакеты uaddr и uaddr2, — и брать их нужно в фиксированном порядке (например, по возрастанию адреса бакета), иначе два встречных requeue заблокируют друг друга. Если оба адреса попали в один бакет, блокировка берётся один раз.
2. Перевешенный поток спит, но его ключ изменился — он теперь в другом бакете. Вспомните, что последний шаг futex_wait — вынуть себя из очереди под блокировкой бакета (раздел 4.8). Куда теперь идти? Самоизвлечение превращается в цикл: прочитать текущий ключ, взять блокировку его бакета, убедиться, что ключ не изменился, пока мы её брали, и только тогда вынимать себя.
3. Условной переменной нужно знать мьютекс, к которому перевешивать: запоминайте его в cond_wait. И главное — разбуженный «через мьютекс» поток должен захватывать мьютекс так, чтобы слово стало 2 (раздел 6.3), иначе его unlock никого не разбудит, и перевешенные потоки останутся в очереди мьютекса навсегда. Поэтому задание требует мьютекса на трёх состояниях.

Проверка — тот же herdbcast: с requeue он должен показать один повторный сон вместо восьми и requeued=7.

8. Семафор

8.1. Определение

Семафор — старейший из примитивов синхронизации: Эдсгер Дейкстра описал его в 1965 году, задолго до появления мьютексов и условных переменных в их современном виде. **Семафор** — это неотрицательный целочисленный счётчик с двумя операциями:

- **wait** (у Дейкстры P, от нидерландского proberen — «пытаться»): уменьшить счётчик на единицу; если счётчик равен нулю — уснуть, пока уменьшение не станет возможным;
- **post** (V, verhogen — «увеличивать»): увеличить счётчик на единицу и, если есть спящие, разбудить одного из них.

Инвариант прост: значение семафора никогда не опускается ниже нуля, а поток, успешно завершивший wait, гарантированно «забрал» одну единицу, добавленную кем-то через post или заложенную при инициализации.

Принципиальное отличие семафора от мьютекса — отсутствие владельца. Мьютекс отпирает тот же поток, который его запер; семафор же может увеличить любой поток, в том числе никогда не вызывавший wait. Это делает семафор средством не только взаимного исключения, но и сигнализации: один поток сообщает другому, что событие произошло. Принципиальное отличие от условной переменной — семафор хранит состояние: post, сделанный, когда никто не ждёт, не теряется, а накапливается в счётчике и будет «съеден» следующим wait.

Семафор с начальным значением 1 ведёт себя как мьютекс и называется бинарным; семафор с произвольным начальным значением называется счётным и выражает идею «в системе есть N единиц ресурса». Обратная сторона универсальности — лёгкость злоупотребления: ни компилятор, ни библиотека не могут проверить, что wait и post образуют осмысленные пары, и дисциплина использования целиком на совести программиста.

8.2. Паттерны

Аллен Дауни в «The Little Book of Semaphores» — основном чтении этого раздела — рассматривает семафор как конструктор, из которого собираются десятки паттернов синхронизации. Нам понадобятся три.

Свидание (rendezvous, Дауни §3.3)

Каждый из двух потоков должен дождаться другого в назначенной точке. Решение использует два семафора aArrived и bArrived с начальным значением 0. Поток A, дойдя до точки встречи, выполняет post(aArrived), затем wait(bArrived); поток B — зеркально. Каждый сначала объявляет о своём прибытии и лишь потом ждёт партнёра. Поучительна ошибочная перестановка: если оба потока сначала ждут, а потом сигналят, получается взаимная блокировка — A ждёт события от B, которое B опубликует только после события от A. Это простейший пример дедлока из циклического ожидания. Заметьте также, чего в решении нет: мьютекса, разделяемого предиката, цикла while — семафор сам хранит состояние «событие произошло».

Мультиплекс (Дауни §3.5)

Семафор с начальным значением K пропускает в секцию не более K потоков одновременно — например, ограничивает число одновременных соединений с базой данных. Каждый

поток выполняет wait перед входом и post после выхода. Мьютексом такое поведение не выражается вовсе, а через условную переменную потребовало бы явного счётчика и цикла.

Производитель-потребитель (Дауни §4.1)

Кольцевой буфер на NBUF элементов охраняют два счётных семафора: empty с начальным значением NBUF считает свободные ячейки, full с начальным значением 0 — заполненные. Производитель выполняет wait(empty), кладёт элемент, затем post(full); потребитель зеркален. Мьютекс по-прежнему нужен, если производителей или потребителей несколько: индексы головы и хвоста у них общие. Сравните с решением на условных переменных из раздела 7: там предикаты count == 0 и count == NBUF проверялись явно в циклах while, здесь их «помнят» счётчики семафоров.

8.3. Интерфейс библиотеки

Библиотека usem объявляет тип struct usem и четыре функции: sem_init(s, value) задаёт начальное значение, sem_wait(s) и sem_post(s) реализуют P и V, sem_trywait(s) — неблокирующий вариант wait: возвращает 0 при успешном уменьшении счётчика и −1, если счётчик равен нулю (поток при этом не засыпает). Функции sem_timedwait нет: наш FUTEX_WAIT не поддерживает таймаутов. Нет и sem_destroy: наши семафоры не владеют никакими ресурсами — это просто слова в памяти пользователя.

За struct usem стоит одна из двух реализаций, которые мы сейчас разберём; какая именно — определяет макрос SEM_ADVANCED в user/usem.h.

8.4. Базовый уровень: семафор из мьютекса и условной переменной

Первая реализация собирается из готовых деталей и служит эталоном корректности. Это ровно канонический шаблон из раздела 7.1, где предикат — value > 0:

```
struct csem {
  struct mutex m;
  struct cond  cv;
  unsigned     value;
};

void
csem_wait(struct csem *s)
{
  mutex_lock(&s->m);
  while (s->value == 0)
    cond_wait(&s->cv, &s->m);
  s->value--;
  mutex_unlock(&s->m);
}

void
csem_post(struct csem *s)
{
  mutex_lock(&s->m);
  s->value++;
  cond_signal(&s->cv);      // post добавил одну единицу - будим одного
  mutex_unlock(&s->m);
}
```

csem_trywait устроен как csem_wait, но вместо цикла ожидания при нулевом счётчике возвращает −1. Корректность этой конструкции следует прямо из правила использования условной переменной: цикл while защищает от ложных и «чужих» пробуждений, мьютекс делает проверку и изменение счётчика атомарными. signal, а не broadcast, — потому что каждый post добавляет ровно одну единицу и будить больше одного ожидающего бессмысленно.

Плата за простоту — каждый вызов, включая беспрепятственный sem_wait при ненулевом счётчике, проходит через полный протокол мьютекса, а каждый post безусловно зовёт cond_signal, то есть FUTEX_WAKE, даже если никто не ждёт. Для семафора, который по смыслу и есть один счётчик, это выглядит расточительно.

8.5. Продвинутый уровень: семафор прямо на futex

Идея прямой реализации — хранить счётчик как слово, над которым работают атомарные операции и futex. По существу так устроен sem_t в glibc. Структура состоит из двух полей:

```
struct fsem {
  unsigned value;      // счётчик; это же слово - ключ futex
  unsigned nwaiters;   // число потоков в медленном пути fsem_wait
};
```

Быстрый путь: CAS-цикл

Забрать единицу — значит атомарно уменьшить положительный счётчик. Одного CAS тут мало:

```
static int
try_dec(struct fsem *s)
{
  unsigned v = s->value;
  while (v > 0) {
    if (__atomic_compare_exchange_n(&s->value, &v, v - 1, 0,
                                    __ATOMIC_SEQ_CST, __ATOMIC_SEQ_CST))
      return 0;               // забрали единицу
    // не вышло: CAS уже записал в v свежее значение
  }
  return -1;                  // счётчик пуст
}
```

Обратите внимание на отличие от мьютекса: там CAS был один (0→1), здесь цикл обязателен. Между чтением и CAS другой поток мог изменить счётчик с 5 на 4, и наша попытка CAS(5→4) провалится, хотя единицы ещё есть, — нужно перечитать и повторить. Перечитывать вручную, правда, не придётся: при неудаче __atomic_compare_exchange_n сама записывает в v текущее значение счётчика (раздел 2.2). Если же наблюдаемое значение — ноль, быстрый путь закончился неудачей.

Медленный путь и post

```
void
fsem_wait(struct fsem *s)
{
  if (try_dec(s) == 0)
    return;                                   // быстрый путь
  __atomic_fetch_add(&s->nwaiters, 1, __ATOMIC_SEQ_CST);  // заявить о себе
  for (;;) {
    if (try_dec(s) == 0) {                    // финальная перепроверка
      __atomic_fetch_sub(&s->nwaiters, 1, __ATOMIC_SEQ_CST);
      return;
    }
    futex((volatile int *)&s->value, FUTEX_WAIT, 0);
  }
}

void
fsem_post(struct fsem *s)
{
  __atomic_fetch_add(&s->value, 1, __ATOMIC_SEQ_CST);
  if (s->nwaiters > 0)                        // будим, только если ждут
    futex((volatile int *)&s->value, FUTEX_WAKE, 1);
}
```

Ради проверки в fsem_post и существует nwaiters: в типичной программе большинство post происходит, когда никто не ждёт, и один атомарный инкремент без системного вызова — вся цена операции. Порядок операций в медленном пути существенен и является типичным

местом ошибок: nwaiters увеличивается до финальной перепроверки value, а уменьшается только после успешного захвата.

Не теряется ли пробуждение?

Это главный вопрос раздела, и рассуждение стоит провести для обоих порядков событий.

Первый порядок: потребитель в медленном пути уже увеличил nwaiters, увидел value == 0 и вот-вот вызовет FUTEX_WAIT, но ещё не вызвал. В этот момент производитель выполняет post целиком: увеличивает value до 1, видит nwaiters > 0 и делает FUTEX_WAKE. Будить некого — потребитель ещё не заснул, wake уходит в пустоту. Затем потребитель вызывает FUTEX_WAIT(&value, 0) — и засыпает навсегда при value == 1? Нет: ядро под блокировкой сверяет текущее значение слова с ожидаемым и, обнаружив 1 вместо 0, немедленно возвращает FUTEX_AGAIN. Цикл медленного пути перепроверяет счётчик, CAS забирает единицу.

Второй порядок: post читает nwaiters **до** того, как потребитель его увеличил, видит 0 и никого не будит. Безопасно ли это? Раз потребитель ещё не увеличил nwaiters, он ещё не выполнил и финальную перепроверку value: она стоит в коде позже, а порядок __ATOMIC_SEQ_CST не даёт процессору переставить их. Инкремент value в post тоже произошёл раньше чтения nwaiters. Значит, к моменту финальной перепроверки потребитель увидит уже увеличенное значение и заберёт единицу без сна.

Проследите обе цепочки внимательно: это ровно тот стиль анализа перекрёстных исполнений, которым Дреппер доказывает корректность своих мьютексов, и в отчёте его нужно воспроизвести своими словами. А потом сдвиньте в fsem_wait инкремент nwaiters на позицию после финальной перепроверки и найдите чередование, при котором поток засыпает навсегда.

Наивный post

Первое, что приходит а ум — выбросить nwaiters и делать FUTEX_WAKE на каждом post. Корректность не пострадает, но каждый post станет системным вызовом. Разоблачается наивный вариант тем же инструментом, что и мьютекс на двух состояниях: счётчиком wake_calls. Параллель полная, и это не совпадение: «не ходи в ядро, когда никто не ждёт» — сквозной принцип всех futex-конструкций.

Упомянем альтернативную схему, которую реализовывать не будем: хранить в одном слове и значение, и число ожидающих, договорившись, что отрицательное значение −k означает «ноль единиц и k ожидающих». Схема изящна, но значение семафора перестаёт быть значением семафора — прочитать «сколько единиц доступно» уже нельзя без интерпретации знака. glibc в итоге тоже живёт с парой value/nwaiters (на 64-битных платформах упакованной в одно 64-битное слово), так что наш дизайн — не учебное упрощение, а уменьшенная копия боевого.

8.6. Дополнительное задание: многоразовый барьер

Барьер — точка программы, которую ни один из N потоков не может пройти, пока к ней не пришли все N; свидание из раздела 8.2 — его частный случай для двух участников. Одноразовый барьер собирается из мьютекса, счётчика пришедших и семафора-вертушки (turnstile) с начальным значением 0: каждый поток под мьютексом увеличивает счётчик, последний пришедший делает post вертушки, а затем все выполняют wait, немедленно за которым следует post, — пробуждения каскадом прокатываются по всем ожидающим.

Трудность начинается, когда барьер должен работать в цикле, многократно: наивное обнуление счётчика позволяет быстрому потоку убежать на следующую итерацию и пройти барьер повторно, пока медленные ещё не покинули предыдущий. Каноническое решение Дауни — двухфазный барьер с двумя вертушками (§3.7): первая заперта, пока не соберутся все, вторая — пока все не покинут первую фазу; потоки, таким образом, не могут перемешаться между итерациями. Реализуйте его поверх struct usem и докажите в отчёте невозможность обгона.

9. Блокировка чтения-записи

9.1. Задача читателей и писателей

Мьютекс, семафор и условная переменная решают задачу взаимного исключения в её самой строгой постановке: в критической секции находится не более одного потока, и неважно, что он там делает. Между тем во многих реальных сценариях такая строгость избыточна. Рассмотрим разделяемую структуру данных — таблицу маршрутизации, кэш, конфигурацию, — которую подавляющее большинство потоков только читает и лишь изредка кто-то модифицирует. Два читателя не мешают друг другу: чтение не меняет состояния. Опасны только комбинации с участием записи: писатель, работающий одновременно с читателем, может показать тому полуобновлённую структуру, а два одновременных писателя способны разрушить её совсем. Обычный мьютекс корректен, но расточителен: он сериализует читателей, которым сериализация не нужна.

Эта постановка известна как задача читателей и писателей. Требуемые инварианты:

- читателей внутри может быть сколько угодно, но только пока внутри нет писателя;
- писатель внутри всегда один и всегда в одиночестве.

Примитив, обеспечивающий эти инварианты, называется блокировкой чтения-записи (rwlock). У него два режима захвата — разделяемый (читательский) и исключительный (писательский). Вся содержательная сложность раздела будет не в том, как обеспечить инварианты, — это, как мы увидим, почти тривиально, — а в том, кого пускать первым, когда у двери стоят и те, и другие.

Интерфейс

Библиотека urwlock предоставляет пять операций: rwlock_init, rwlock_rdlock и rwlock_rdunlock для читательской стороны, rwlock_wrlock и rwlock_wrunlock для писательской. В POSIX pthread_rwlock_t освобождается единой функцией pthread_rwlock_unlock независимо от режима захвата. Такая унификация удобна вызывающему, но реализация тогда обязана сама помнить, в каком режиме замок захвачен. Мы перекладываем это знание на вызывающего — и реализация становится ровно такой, какой её диктуют инварианты.

Строится библиотека поверх мьютекса и условных переменных: мьютекс защищает внутреннее состояние rwlock, условные переменные обслуживают ожидание. Это осознанный выбор уровня. Приёмы прямой работы с futex — CAS-циклы, возврат FUTEX_AGAIN, счётчик ожидающих — мы отработали в мьютексе и семафоре. Новизна этого раздела лежит выше по стеку: правильность реализации здесь впервые вопрос не только безопасности, но и живости (раздел 1.2).

9.2. Наивное решение

Первая версия, rwlock1, кодирует инварианты самым прямым способом. Состояние — два поля под защитой мьютекса: nreaders, число читателей внутри, и флаг writer.

```
struct rwlock1 {
  struct mutex lk;          // защищает все поля ниже
  struct cond c;            // общая очередь: спят и читатели, и писатели
  int nreaders;             // читателей внутри
  int writer;               // 1, если внутри писатель
  unsigned rd_acquires;     // всего читательских захватов (для замера)
};

void
rwlock1_rdlock(struct rwlock1 *rw)
{
  mutex_lock(&rw->lk);
  while(rw->writer)                    // ждём только писателя ВНУТРИ
    cond_wait(&rw->c, &rw->lk);
  rw->nreaders++;
  rw->rd_acquires++;
  mutex_unlock(&rw->lk);
}

void
rwlock1_rdunlock(struct rwlock1 *rw)
{
  mutex_lock(&rw->lk);
  rw->nreaders--;
  if(rw->nreaders == 0)
    cond_broadcast(&rw->c);
  mutex_unlock(&rw->lk);
}

void
rwlock1_wrlock(struct rwlock1 *rw)
{
  mutex_lock(&rw->lk);
  while(rw->writer || rw->nreaders > 0)
    cond_wait(&rw->c, &rw->lk);
  rw->writer = 1;
  mutex_unlock(&rw->lk);
}

void
rwlock1_wrunlock(struct rwlock1 *rw)
{
  mutex_lock(&rw->lk);
  rw->writer = 0;
  cond_broadcast(&rw->c);
  mutex_unlock(&rw->lk);
}
```

Ожидание в обоих случаях — канонический цикл while по предикату. Осталось решить, кого и как будить при освобождении. И здесь версия с одной условной переменной наталкивается на принципиальное ограничение: на одной очереди спят ожидающие двух разных сортов с разными предикатами. Уходящий писатель не знает, кто спит первым в очереди — читатель или другой писатель, — и потому signal небезопасен: сигнал может достаться потоку, чей предикат всё ещё ложен; тот проверит условие, снова заснёт — и сигнал пропал, а поток, который мог бы пройти, спит дальше. Сравните с семафором: там все спящие были одинаковы, любой разбуженный годился, и signal был точен. Здесь разнородность ожидающих вынуждает broadcast — разбудить всех и дать каждому перепроверить свой предикат. Цену этого решения мы уже знаем по разделу 7.3.

Но стадо — не главный дефект rwlock1. Главный дефект не виден в коде вовсе; он виден только под нагрузкой.

9.3. Голодание писателя

Проследим за писателем при плотном потоке читателей (рисунок 10, верхняя часть). Писатель пришёл, увидел nreaders равным, скажем, трём и заснул ждать нуля. Пока эти трое дочитывают, приходят новые читатели. Предикат читателя проверяет только флаг writer, а флаг не поднят — писатель ведь ещё не внутри, он только ждёт. Новые читатели входят беспрепятственно, nreaders колеблется, но при достаточно частых приходах никогда не касается нуля. Писатель ждёт неограниченно долго, хотя замок ни секунды не простаивал и все остальные исправно работали.



Рисунок 10. Голодание писателя при reader-preference и его устранение при writer-preference

Подчеркнём, что это дефект нового для нас сорта. Все ошибки предыдущих разделов — гонка на инкременте, потерянный сигнал — были нарушениями безопасности: система могла прийти в противоречивое состояние. Здесь безопасность безупречна: инварианты rwlock1 соблюдаются в каждый момент времени, тест на разрывы чтения пройдёт с любыми числами. Нарушена живость — третье требование из раздела 1.2, ограниченное ожидание: конкретному потоку не гарантирован прогресс. Неограниченное откладывание потока при формально корректной работе системы называется голоданием (starvation), а политика rwlock1, при которой читатели проходят всегда, пока внутри нет писателя, называется reader-preference.

9.4. Измерение голодания

Прежде чем чинить, научимся измерять — иначе починку нечем будет подтвердить. Напрашивающаяся метрика, время ожидания писателя, для xv6 неудачна: часы грубые, а планировщик недетерминирован, и числа будут плавать от прогона к прогону. Возьмём метрику дискретную и воспроизводимую: число читательских захватов, совершённых за время ожидания писателя — от момента, когда писатель запросил замок, до момента, когда он его получил. Эта величина напрямую пересчитывает само явление: голодание писателя и есть поток читателей, прошедших мимо него без очереди.

Технически для неё достаточно поля rd_acquires, которое увеличивается при каждом успешном читательском захвате. Инкремент происходит под внутренним мьютексом,

который в этот момент и так захвачен, поэтому инструментирование бесплатно и не вносит гонок. Вся логика измерения живёт в тесте: писатель снимает значение счётчика перед вызовом rwlock_wrlock, снимает после возврата и берёт разность. У наивной версии счёт идёт на сотни.

9.5. Продвинутый уровень: writer-preference

Починка голодания — это смена политики: читатель должен уступать не только писателю внутри, но и писателю у двери. Для этого замку нужно знать о стоящих у двери писателях. Как только писатель заявил о себе, приток новых читателей перекрывается, наличные читатели дочитывают, их число монотонно убывает до нуля, и писатель входит (рисунок 10, нижняя часть). Такая политика называется writer-preference.

Реализуйте rwlock2 с этой политикой самостоятельно. Классическое решение на семафорах разобрано у Дауни (§4.2, writer-priority readers-writers); ваша задача — перенести его идею на мьютекс и условные переменные. При этом заодно почините и механику побудки: разведите читателей и писателей по разным условным переменным, чтобы освобождающий знал, кого будит, и мог будить адресно. Продумайте и обоснуйте в отчёте:

- какое поле нужно добавить в состояние и где именно (до или после ожидания) его менять;
- как меняется предикат читателя и предикат писателя;
- кого будит последний уходящий читатель и кого — уходящий писатель; где достаточно signal, а где нужен broadcast. Подсказка: правило «будить стольких, скольких событие способно пропустить» из раздела 7.1. Уход писателя при отсутствии других писателей пропускает всех ждущих читателей сразу — и broadcast здесь впервые за работу оказывается не ценой простоты, а семантически точным инструментом: разбуженные читатели совместимы друг с другом и войдут все;
- не заголодают ли теперь читатели и почему для типичной нагрузки rwlock это приемлемо.

Критерий приёмки — тот же тест на голодание: разность rd_acquires за ожидание писателя не должна превышать числа читателей, успевших войти до заявки.

9.6. За пределами работы

Существуют политики честнее writer-preference. Ticket-подходы переносят на rwlock идею очереди по номеркам из алгоритма пекарни: захватчики обслуживаются строго в порядке прихода, голодание исключено для всех. Phase-fair-замки чередуют читательские и писательские фазы, гарантируя, что писатель ждёт не более одной читательской фазы, а читатель — не более одного писателя; это даёт доказуемые оценки ожидания и делает такие замки стандартом в системах реального времени (Brandenburg, Anderson, «Spin-Based Reader-Writer Synchronization for Multiprocessor Real-Time Systems»). Наконец, rwlock можно построить прямо над futex, упаковав число читателей, флаг писателя и флаг ожидающих в одно слово, — так устроен pthread_rwlock в glibc. После этой работы у вас есть все приёмы, чтобы разобрать его исходный код.

10. Практическая часть

10.1. Подготовка

Что мы строим

Работа выполняется в дереве xv6 из лабораторной работы 1: потоки, clone, thread_join и библиотека uthread должны работать, тесты threadtest, racetest и benchtest — проходить. Практическая часть разбита на шесть блоков, каждый заканчивается контрольной точкой — набором тестов, которые должны пройти, прежде чем двигаться дальше. Блоки идут в порядке зависимостей: каждый следующий опирается на предыдущий.

Блок	Что реализуете	Файлы	Тесты
1. Futex	системный вызов futex и счётчики futexstats	kernel/futex.c, kernel/proc.h, обвязка системного вызова	futextest
2. Join	механизм ctid, thread_join_t поверх futex	kernel/proc.c, kernel/sysproc.c, user/uthread.c	herdtest
3. Мьютекс	мьютекс на futex	user/umutex.c	mutextest
4. Условная переменная	cond_wait, cond_signal, cond_broadcast	user/umutex.c	prodcons, herdbcast
5. Семафор	семафор и его тесты	user/usem.c	prodcons_sem, sempool, naivepost
6. rwlock	блокировка чтения-записи	user/urwlock.c	rwtest, wrstarve

Уровни сложности

Каждый блок имеет базовый и продвинутый уровень. Какой уровень вы выполняете, определяет преподаватель. Продвинутый уровень включает базовый: реализация базового уровня остаётся в коде и участвует в сравнительных замерах. Переключаются уровни макросами в заголовочных файлах — так одни и те же тесты работают на обоих уровнях:

Блок	Базовый уровень	Продвинутый уровень	Макрос
1. Futex	глобальная блокировка + sleep/wakeup (раздел 4.7)	хеш-таблица очередей, WAKE(n) (раздел 4.8)	FUTEX_ADVANCED в kernel/futex.c
2. Join	механизм ctid	то же	—
3. Мьютекс	два состояния, xchg (раздел 6.2)	Take 1, 2, 3 по статье Дреппера + спин (раздел 6.3)	MUTEX_ADVANCED в user/umutex.h
4. Условная переменная	реализация из раздела 7.2	то же	—
5. Семафор	csem: мьютекс + condvar (раздел 8.4)	fsem: прямо на futex, naivepost (раздел 8.5)	SEM_ADVANCED в user/usem.h
6. rwlock	rwlock1 и замер голодания (разделы 9.2–9.4)	rwlock2, writer-preference (раздел 9.5)	RWLOCK_ADVANCED в user/urwlock.h

Дополнительные задания (раздел 10.8) выполняются по желанию и оцениваются отдельно.

Как запускать тесты и читать замеры

Каждый тест — обычная программа xv6: соберите систему (make qemu) и запустите тест из оболочки по имени. Программа runtests прогоняет все тесты подряд и подводит итог; на базовом уровне запускайте её с аргументом basic:

```
$ runtests basic
...
runtests: 15/15 passed
```

Тест считается пройденным, если он завершился с кодом 0. Большинство тестов, кроме проверки корректности, печатают замеры — строки вида

```
prodcons: slept=6188 again=1657 woken=6188 wake_calls=22740
```

Это разность счётчиков futexstats до и после сценария (раздел 5.2). Числа зависят от планировщика и будут меняться от запуска к запуску; ниже в контрольных точках приведены эталонные значения, полученные в QEMU с тремя процессорами (CPUS := 3 в Makefile). Если не сказано иное, эталон снят на продвинутом уровне; на базовом уровне futex счётчик woken всегда равен нулю, а времена заметно больше. Расхождение на десятки процентов — норма, важны порядок величин и соотношения между версиями. Для отчёта запускайте каждый замер не менее трёх раз.

Дисциплина пересборки. После правок в заголовочных файлах (proc.h, futex.h, umutex.h и других) и после переключения макросов уровня собирайте через make clean && make qemu. Система зависимостей xv6 отслеживает изменения заголовков не полностью, и инкрементальная сборка может оставить в fs.img старые бинарники.

10.2. Блок 1. Системный вызов futex

Шаг 1: каркас системного вызова

Начнём с механики, знакомой по лабораторной работе 1: новый системный вызов нужно провести через все слои. Скопируйте kernel/futex.h и изучите комментарии — он важнее кода (раздел 4.3).

Дальше по накатанной: номера SYS_futex и SYS_futexstats в kernel/syscall.h, объявления и записи в таблице syscalls в kernel/syscall.c, entry() в user/usys.pl, прототипы в user/user.h:

```
struct futexstat;
int futex(volatile int *uaddr, int op, int val);
int futexstats(struct futexstat *s);
```

Обработчик sys_futex в kernel/sysproc.c — простой диспетчер: извлечь три аргумента (argaddr, argint, argint) и по op вызвать futex_wait(uaddr, val) или futex_wake(uaddr, val); для неизвестной операции и отрицательного n вернуть −1. Сами futex_wait и futex_wake будут жить в новом файле kernel/futex.c — добавьте $K/futex.o в OBJS и объявите функции в kernel/defs.h. Функцию futexinit() вызовите из main() до userinit().

Обработчик sys_futexstats копирует глобальную struct futexstat пользователю через copyout. Счётчики увеличивайте атомарно — __atomic_fetch_add(&futexstat.wait_slept, 1, __ATOMIC_SEQ_CST) — без отдельной блокировки (раздел 5.2).

Не забывайте включать futex.h во все файлы, где используются константы или struct futexstat: компилятор напомнит ошибками «undeclared» и «incomplete type».

Шаг 2: ключ фьютекса

Реализуйте futex_key и futex_load по разделу 4.6. Проверить этот путь целиком позволяет тест 3a из futextest: sbrk(PGSIZE) без единого касания страницы, затем FUTEX_WAIT по этому адресу с заведомо ненулевым val — ядро обязано молча догрузить страницу, прочитать из неё ноль и вернуть FUTEX_AGAIN.

Шаг 3a: базовый уровень

Реализуйте futex_wait и futex_wake на глобальной блокировке и sleep/wakeup (раздел 4.7). Не забудьте счётчики: wait_again при возврате FUTEX_AGAIN, wait_slept перед sleep, wake_calls в каждом futex_wake.

Шаг 3б: продвинутый уровень

Реализуйте хеш-таблицу бакетов, поля futex_key и futex_next в struct proc (обнулять в freeproc), развёрнутый sleep в futex_wait, адресное пробуждение в futex_wake и самоизвлечение из очереди после sched() (раздел 4.8). Счётчик woken увеличивайте за каждый поток, реально переведённый из SLEEPING в RUNNABLE. Базовую реализацию оставьте в том же файле под #ifndef FUTEX_ADVANCED: она понадобится для сравнения.

Чекпоинт 1

Базовый уровень — тесты 1–5 обязаны проходить:

```
$ futextest basic
unaligned addr rejected: OK
value mismatch -> FUTEX_AGAIN: OK
wake with no waiters -> 0: OK
lazily allocated page: OK
wake sleeping thread: OK
sleeper joined: OK
no lost wakeup: OK
futextest: basic tests passed
```

Запустите на базовом уровне и полный futextest: тест 6 упадёт, показав «woken after WAKE(1): 4 of 4». Это ожидаемо (wakeup будит всех) — сохраните вывод для отчёта.

Продвинутый уровень:

```
$ futextest
...
woken after WAKE(1): 1 of 4 (advanced: 1, basic: 4)
WAKE(1) wakes exactly one: OK
WAKE(n) released the rest: OK
futextest: all tests passed
```

Если вместо этого

Тест 4 или 5 зависает. Скорее всего, потеряно пробуждение: проверьте, что проверка значения и постановка в очередь (или sleep) выполняются под той же блокировкой, которую берёт futex_wake, и что в продвинутой версии p->lock захватывается до release(&b->lock).

panic: acquire или panic: sched locks. Нарушен порядок блокировок или какая-то блокировка не отпущена на одном из путей возврата. Пройдитесь по всем return в futex_wait.

Тест «lazily allocated page» возвращает −1. futex_key не догружает страницу: после неудачного walkaddr должен идти vmfault.

10.3. Блок 2. thread_join поверх futex

Шаг 1: ядро

Добавьте в struct proc поле uint64 ctid, обнуляйте его в allocproc и freeproc. Расширьте clone четвёртым параметром — адресом ctid-слова — и сохраняйте его в np->ctid. Обновите сигнатуру согласованно во всех местах: kernel/defs.h, sys_clone в kernel/sysproc.c и прототип в user/user.h:

```
int clone(void (*fn)(void*), void *arg, void *stack, int *ctid);
```

Старые вызовы с тремя аргументами оставили бы в регистре a3 мусор, поэтому меняйте всё сразу. В kexit — после закрытия файлов и до захвата wait_lock — добавьте запись нуля и futex_wake (раздел 4.9). Вызов wakeup((void*)&proc) в kexit не удаляйте.

Шаг 2: библиотека uthread

Добавьте в таблицу потоков поле volatile int alive. В thread_create записывайте alive = 1 до clone и передавайте &slot->alive четвёртым аргументом. В thread_join_t сначала ждите обнуления alive в цикле с FUTEX_WAIT, затем вызывайте thread_join(tid) для уборки записи ZOMBIE, затем освобождайте стек.

Шаг 3: счётчик join_sleeps

В системном вызове thread_join перед каждым sleep увеличивайте futexstat.join_sleeps — так мы сможем посчитать стадо.

Чекпоинт 2

Тест herdtest создаёт 8 рабочих потоков, завершающихся по очереди, и 8 ожидающих, каждый ждёт своего рабочего. В режиме old ожидающие сидят внутри системного вызова thread_join на общем канале, в режиме futex — в FUTEX_WAIT на своих ctid-словах:

```
$ herdtest old
herdtest: 8 workers, 8 joiners, mode = old (shared channel)
  sleeps in thread_join (herd):  44
  futex: slept=0 again=0 woken=0 wake_calls=16 join_sleeps=44
$ herdtest futex
herdtest: 8 workers, 8 joiners, mode = futex
  sleeps in thread_join (herd):  0
  futex: slept=16 again=0 woken=16 wake_calls=16
```

Обратите внимание на характер чисел. На общем канале каждый exit будит всех ещё спящих ожидающих, и суммарно получается порядка $N + (N-1) + \ldots + 1 = N(N+1)/2 = 36$ засыпаний плюс несколько лишних; результат недетерминирован — планировщик решает, сколько ожидающих успело уснуть к очередному exit. На futex числа детерминированы: 16 засыпаний (восемь ожидающих плюс восемь ожиданий главного потока) и ровно 16 пробуждений, одно событие — одно пробуждение. На базовом уровне futex режим futex тоже покажет 0 засыпаний в join (у каждого слова свой канал — стадо исчезает и там), но woken останется нулём: wakeup не умеет отчитываться.

Регрессия: threadtest, racetest и benchtest из лабораторной работы 1 обязаны работать без изменений.

10.4. Блок 3. Мьютекс

Шаг 1: базовый уровень

Скопируйте user/umutex.h. В user/umutex.c реализуйте mutex_init, mutex_xchg_lock и mutex_xchg_unlock (раздел 6.2), а основной интерфейс mutex_lock/mutex_unlock на базовом уровне сделайте обёртками над ними:

```
void mutex_lock(struct mutex *m)   { mutex_xchg_lock(m); }
void mutex_unlock(struct mutex *m) { mutex_xchg_unlock(m); }
```

Всё, что будет построено дальше — условные переменные, семафоры, rwlock, — пользуется только mutex_lock и mutex_unlock. Переключив реализацию здесь, вы переключите её во всех примитивах сразу.

Шаг 2: продвинутый уровень

Прочитайте разделы 4–6 статьи Дреппера «Futexes Are Tricky». Реализуйте функции, объявленные в umutex.h под MUTEX_ADVANCED: drepper1_lock/drepper1_unlock (Take 1), drepper2_lock (Take 2), drepper3_lock (Take 3), drepper_spin_lock (Take 3 со спином перед

сном) и общий drepper_unlock для трёх последних. Основной интерфейс переключите на Take 3.

Чекпоинт 3

Тест mutextest проверяет каждый вариант двумя сценариями. В первом один поток 5000 раз захватывает и освобождает мьютекс без конкуренции. Во втором шесть потоков по 10000 раз увеличивают общий счётчик под мьютексом; итог обязан совпасть с 60000 (иначе тест печатает FAIL), а сценарий прогоняется с короткой и с длинной критической секцией. Базовый уровень:

```
$ mutextest
--- uncontended ---
ticks=5  xchg     solo : slept=0 again=0 woken=0 wake_calls=5000
--- contended, short cs ---
ticks=58  xchg     short: slept=10 again=2 woken=0 wake_calls=60030
--- contended, long cs ---
ticks=55  xchg     long : slept=82 again=16 woken=0 wake_calls=60030
mutextest: OK
```

Это вывод на базовом уровне futex. Продвинутый уровень добавляет в сравнение варианты Дреппера (вывод сокращён):

```
--- uncontended ---
ticks=2  xchg     solo : slept=0 again=0 woken=0 wake_calls=5000
ticks=2  drepper1 solo : slept=0 again=0 woken=0 wake_calls=5000
ticks=0  drepper2 solo : slept=0 again=0 woken=0 wake_calls=0
ticks=0  drepper3 solo : slept=0 again=0 woken=0 wake_calls=0
ticks=0  spin     solo : slept=0 again=0 woken=0 wake_calls=0
--- contended, long cs ---
ticks=10  xchg     long : slept=79 again=3786 woken=79 wake_calls=60006
ticks=12  drepper1 long : slept=56 again=3131 woken=56 wake_calls=60006
ticks=2  drepper2 long : slept=16 again=2606 woken=16 wake_calls=3240
ticks=2  drepper3 long : slept=13 again=646 woken=13 wake_calls=907
ticks=1  spin     long : slept=10 again=1482 woken=10 wake_calls=2685
```

Главное, что нужно увидеть и объяснить в отчёте: у вариантов на двух состояниях (xchg и drepper1) wake_calls равен числу освобождений — каждый unlock ходит в ядро, — а у вариантов на трёх состояниях без конкуренции он равен нулю и под конкуренцией на порядок меньше. Отсюда и разница во времени (ticks) в разы. Сравните также время мьютекса xchg на базовом и продвинутом уровне futex (58 и 10 тиков в эталоне): глобальная блокировка и wakeup, обходящий всю таблицу процессов, заметно медленнее бакетов.

10.5. Блок 4. Условная переменная

Шаг 1

Добавьте в user/umutex.c реализацию условной переменной из раздела 7.2 (cond_init, cond_wait, cond_signal, cond_broadcast). Код дан в теоретической части целиком; ваша задача в этом блоке — понять и обосновать его, а также провести замеры.

Чекпоинт 4

Тест prodcons — классическое решение задачи производителей-потребителей на условных переменных: кольцевой буфер на 8 элементов, два производителя по 5000 элементов, три потребителя, сверка контрольных сумм. Для завершения главный поток ставит флаг done и делает broadcast:

```
$ prodcons
prodcons: 2 prod x 5000 items, 3 cons, buf=8
```

```
prodcons: OK ticks=8
prodcons: slept=6188 again=1657 woken=6188 wake_calls=22740
```

Тест herdbcast показывает стадный эффект broadcast (раздел 7.3): восемь потоков спят на условной переменной, главный поток делает один broadcast, продолжая держать мьютекс:

```
$ herdbcast
herdbcast: 8 waiters, one broadcast
cond  : slept=8 again=0 woken=16 wake_calls=18
```

Объясните в отчёте каждое число: откуда 8 пробуждений от broadcast, ещё 8 — по цепочке на мьютексе, и почему slept=8 — это и есть стадо.

10.6. Блок 5. Семафор

Шаг 1: базовый уровень

Скопируйте user/usem.h. В user/usem.c реализуйте csem_init, csem_wait, csem_trywait, csem_post (раздел 8.4) и основной интерфейс sem_* как обёртки над ними. Тесты пользуются только struct usem и функциями sem_*.

Шаг 2: продвинутый уровень

Реализуйте fsem_init, fsem_wait, fsem_trywait, fsem_post и наивный fsem_post_naive (раздел 8.5), переключите основной интерфейс на fsem.

Чекпоинт 5

prodcons_sem — тот же производитель-потребитель на двух счётных семафорах, с «ядовитыми пилюлями» для завершения: главный поток, дождавшись производителей, проталкивает через тот же семафорный протокол по одному специальному значению на каждого потребителя. sempool — мультиплекс: семафор на K = 4 единицы, 12 потоков, проверка, что внутри одновременно не больше K:

```
$ prodcons_sem
prodcons_sem: produced=50005000 consumed=50005000 OK
prodcons_sem: slept=1977 again=1959 woken=1977 wake_calls=17650
$ sempool
sempool: K=4 max_inside=4 OK
```

Сравните замеры prodcons_sem на базовом и продвинутом уровне, а также с prodcons из блока 4. У семафорной версии нет стада от broadcast, а каждый post будит не более одного потока. Есть ли выигрыш по суммарному числу засыпаний и от чего он зависит?

На продвинутом уровне naivepost сравнивает fsem_post и fsem_post_naive на одной нагрузке:

```
$ naivepost
solo      M=5000     nwaiters: slept=0 again=0 woken=0 wake_calls=0
solo      M=5000     naive   : slept=0 again=0 woken=0 wake_calls=5000
contended posts=4000  nwaiters: slept=2 again=0 woken=2 wake_calls=8
contended posts=4000  naive   : slept=3 again=0 woken=3 wake_calls=4008
```

10.7. Блок 6. Блокировка чтения-записи

Шаг 1: базовый уровень

Скопируйте user/urwlock.h. В user/urwlock.c реализуйте rwlock1 (раздел 9.2).

Шаг 2: продвинутый уровень

Реализуйте rwlock2 с политикой writer-preference и двумя условными переменными (раздел 9.5).

Чекпоинт 6

rwtest проверяет корректность: писатели переписывают массив из 16 ячеек поэлементно, нарочно засыпая посередине, а читатели под rdlock проверяют, что все ячейки равны. Дополнительно счётчики «сколько внутри» проверяют сами инварианты rwlock:

```
$ rwtest
rwlock1: torn=0 violations=0 writes=90 OK
rwlock1: slept=94 again=0 woken=94 wake_calls=1299
rwlock2: torn=0 violations=0 writes=90 OK
rwlock2: slept=63 again=0 woken=63 wake_calls=99
```

Убедитесь, что тест действительно ловит ошибки: временно уберите rdlock в читателе и посмотрите на torn.

wrstarve измеряет голодание (раздел 9.4): восемь читателей непрерывно перекрываются, писатель делает четыре захода, и для каждого печатается число читательских захватов, совершённых за время его ожидания:

```
$ wrstarve
rwlock1: starvation per probe: 448 0 0 0  total=448
rwlock1: slept=3 again=0 woken=3 wake_calls=14
rwlock2: starvation per probe: 0 0 0 0  total=0
rwlock2: slept=37 again=1 woken=37 wake_calls=20
rwlock2: OK
```

У rwlock1 первая проба поглощает почти все читательские захваты: писатель ждал, пока читатели не закончили работу совсем. Последующие пробы нулевые, потому что читателей больше нет. У rwlock2 разность на пробу не должна превышать числа читателей, успевших войти до заявки писателя.

10.8. Дополнительные задания

Многоразовый барьер

Реализуйте двухфазный барьер поверх struct usem (раздел 8.6) в функциях barrier_init и barrier_wait в user/barriertest.c. Тест: 8 потоков, 50 раундов; в каждом раунде каждый поток увеличивает свой счётчик, проходит барьер, один назначенный поток проверяет, что все счётчики равны номеру раунда, затем второй барьер:

```
$ barriertest
barriertest: 8 threads, 50 rounds, OK
barriertest: slept=1400 again=1 woken=1400 wake_calls=1608
```

Потокобезопасная uthread

Таблица потоков библиотеки uthread общая для всех потоков процесса, и без защиты два параллельных thread_create могут занять один слот. Защитите таблицу мьютексом. Мьютекс должен покрывать поиск и изменение слотов, но не clone и не ожидание в join: ждать под мьютексом нельзя — остальные потоки не смогли бы ни создавать, ни присоединять потоки. Учтите, что malloc и free из umalloc.c тоже не потокобезопасны. Напишите тест, в котором несколько потоков одновременно создают и присоединяют потоки.

FUTEX_REQUEUE

Реализуйте системный вызов futex_requeue и условную переменную condrq с broadcast через requeue (раздел 7.4). Требует продвинутого уровня блоков 1 и 3. Тест 7 в futextest проверяет системный вызов, herdbcast — эффект:

```
$ herdbcast
```

```
herdbcast: 8 waiters, one broadcast
cond  : slept=8 again=0 woken=16 wake_calls=18
condrq: slept=1 again=0 woken=9 wake_calls=17 requeued=7
```

FUTEX_WAIT с таймаутом

Добавьте четвёртый аргумент — число тиков, по истечении которых FUTEX_WAIT возвращает особый код. Продумайте, кто и как вынимает поток из очереди по таймауту и чем это похоже на пробуждение kill'ом (подсказка: механизм самоизвлечения у вас уже написан). Полная реализация потребует подружить очередь фьютекса с tickslock — оцените сложность, прежде чем браться.

11. Отчёт

Отчёт по работе должен содержать:

1. Описание реализации каждого блока выбранного уровня: ключевые фрагменты кода и обоснование проектных решений. Обязательно своими словами: почему в futex_wait нет окна потерянного пробуждения; инвариант, делающий безопасным разыменование записи из очереди в futex_wake (продвинутый уровень); почему запись нуля в ctid живёт в kexit.
2. Для мьютекса продвинутого уровня — разбор вариантов Дреппера: найденный дефект Take 1, смысл трёх состояний, ответ на вопрос «что означает, если обмен на 2 вернул 0».
3. Для семафора продвинутого уровня — анализ двух порядков событий между fsem_wait и fsem_post (раздел 8.5).
4. Для rwlock продвинутого уровня — обоснование политики и схемы побудки rwlock2 по вопросам раздела 9.5.
5. Сводную таблицу замеров: строки — сценарии (herdtest old/futex, mutextest solo/short/long для каждого варианта, prodcons, herdbcast, prodcons_sem, naivepost, rwtest, wrstarve), столбцы — счётчики slept, again, woken, wake_calls и время в тиках. Для каждой строки — среднее по трём запускам. Под таблицей — интерпретация: какое утверждение теоретической части подтверждает каждая строка.
6. Вывод runtests (или runtests basic).

12. Контрольные вопросы

1. Какие три требования предъявляются к решению задачи о критической секции? Какие из них относятся к безопасности, а какие к живости? Приведите пример нарушения каждого из этой работы.
2. Почему алгоритм Петерсона на современных процессорах может пропустить в критическую секцию два потока сразу?
3. Зачем функции sleep в xv6 второй аргумент — спин-блокировка? Опишите сценарий потерянного пробуждения без него.
4. Почему пользовательскому коду недостаточно пары вызовов «усыпи меня на адресе» / «разбуди на адресе»? Какую проблему решает аргумент val в FUTEX_WAIT?
5. Почему проверка значения в FUTEX_WAIT обязана происходить под той же блокировкой, которую берёт FUTEX_WAKE, и что произойдёт, если вынести её из-под блокировки?
6. Почему FUTEX_WAIT всегда вызывается в цикле? Назовите все причины, по которым он может вернуть управление, когда ожидаемое событие не произошло.

7. Почему ключом фьютекса выбран физический адрес? Что сломается, если использовать пару (pagetable, va), при появлении разделяемой памяти между процессами?
8. Почему запись нуля в ctid и futex_wake живут в kexit, а не в thread_exit пользовательской библиотеки? Зачем после futex-ожидания всё ещё нужен системный вызов thread_join и почему он почти никогда не спит?
9. Почему число засыпаний в herdtest old меняется от запуска к запуску, а фьютексные счётчики в режиме futex — нет?
10. Почему в мьютексе на двух состояниях нет потерянного пробуждения? Почему пробуждение не передаёт владение мьютексом?
11. Что такое перехват (barging) и почему он допустим?
12. Что в мьютексе на трёх состояниях означает ситуация, когда атомарный обмен на 2 вернул 0? Почему спиннер не должен переводить слово в 2?
13. Почему в cond_wait значение seq читается до освобождения мьютекса? Что сломается, если поменять эти строки местами?
14. Почему cond_wait вызывается только в цикле while? Почему условная переменная не хранит состояние, а семафор хранит?
15. В чём причина стадного эффекта cond_broadcast и как его устраняет FUTEX_REQUEUE?
16. Чем семафор отличается от мьютекса и от условной переменной? Приведите задачу, которая естественно решается семафором и неудобно — мьютексом.
17. Почему rwlock1 вынужден использовать broadcast? Почему его тест на корректность проходит, несмотря на голодание писателя?
18. Почему broadcast читателям при уходе писателя в rwlock2 — не стадо?

Литература

1. Franke H., Russell R., Kirkwood M. Fuss, Futexes and Furwocks: Fast Userlevel Locking in Linux. Ottawa Linux Symposium, 2002.
2. Drepper U. Futexes Are Tricky. Red Hat, 2004–2011. Разделы 4–6 (мьютекс), 8 (requeue).
3. Downey A. The Little Book of Semaphores. Главы 3 (базовые паттерны: §3.3 rendezvous, §3.5 multiplex, §3.6–3.7 барьер) и 4 (§4.1 производитель-потребитель, §4.2 читатели-писатели).
4. Cox R., Kaashoek F., Morris R. xv6: a simple, Unix-like teaching operating system. Глава «Locking» и «Scheduling» (sleep и wakeup).